AI Tutor: System Design (Phase 2)

AI Tutor: An Evidence-Based Adaptive Learning System with Deliberate Scaffolding Withdrawal

Course	MCA-269 Minor Project III
Phase	2: System design (DFD, ER, UML) and partial implementation
Date	15 Oct 2026
Team	Ashwani Kumar Puri (04914004425): backend, architecture, database · Aman Dogra (01614004425): frontend · Aditi Tanwar (01814004425): frontend, UI/UX
Guide	*to be filled*
Builds on	Phase 1 SRS (`../phase-1-submitted/MP3-SRS.pdf`), API spec (`../spec/`), ADRs (`../adr/`)

Contents

1. Scope of this document
2. Design decisions
3. Architecture
4. Data flow diagrams
5. Database design and ER diagrams
6. UML diagrams
7. API design
8. Non-functional design
9. Requirement traceability
10. Partial implementation status
11. Open items

---

1. Scope of this document

This document turns the Phase 1 SRS into a buildable design. It covers the whole system (13 modules, ~98 API operations) so nothing has to be redesigned later. It also marks what is built first: the **MVP due 10 Nov 2026**.

Core idea, unchanged from Phase 1: the system keeps a **separate reading and writing mastery score per skill**, updates it **only from executed code**, and **withdraws help on a schedule** until the learner can work alone.

---

2. Design decisions

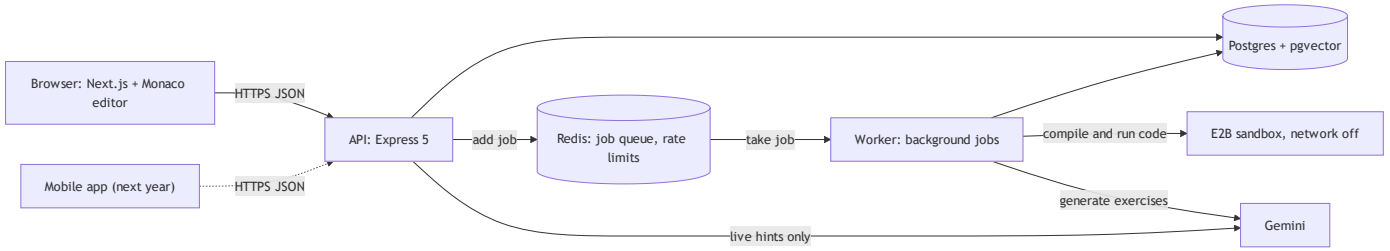
Area	Decision	Source
First course	Data Structures & Algorithms. More courses (frontend, backend, full stack) follow	ADR-0001, 13 Oct
Languages	Engine is language-agnostic. **C++20 (clang 19)** ships first; Java and Python follow	ADR-0002, 13 Oct
Code execution	E2B sandbox (network off) + our own judge adapter; one sandbox reused per learner session	ADR-0003, 13 Oct
Verdicts	AC, WA, CE, RE, TLE, MLE, **SE** (sandbox error: not the learner's fault, no mastery effect)	ADR-0004, 13 Oct
Content	LLM-generated, then **verified in the sandbox and frozen**. Grading and next-step selection never use an LLM	ADR-0005
Exercise delivery	Pre-generated in the background; async job with a status endpoint as fallback	13 Oct
Hint level 5	Shows the frozen reference solution, logged as a bottom-out, no mastery credit for that attempt	13 Oct
Compile errors	After the 3rd on the same exercise: **offer** a hint, never push one (FR-M8-03 stays intact)	13 Oct
TLE/MLE penalty	Starts at 1/20 of the WA penalty, doubles on each consecutive TLE/MLE, capped at the WA penalty, resets on AC	ADR-0004, 13 Oct
Admin panel	Cut from the MVP: skill graph is seeded by script; analytics is one admin-only page	13 Oct

Area	Decision	Source
Auth	JWT in httpOnly cookie for web, `Authorization: Bearer` for the planned mobile app	13 Oct
Database	PostgreSQL 16 + pgvector, schema and migrations with Drizzle ORM	13 Oct
LLM	Google Gemini behind one adapter (small model for hints, large model for generation)	13 Oct
Queue	Redis + BullMQ, single Redis client (`ioredis`)	13 Oct

ADR-0002, ADR-0004 and ADR-0006 will be updated to reflect the 13 Oct decisions.

---

3. Architecture



Component	Responsibility
Web (`apps/web`)	Learner and admin screens, Monaco code editor
API (`apps/api`)	Auth, validation, all 13 modules, fast reads and writes
Worker (`apps/api`, second entry point)	Exercise generation and verification, code execution, scheduled jobs (decay scan, fade evaluation, pre-generation)
PostgreSQL	Single source of truth for all data
Redis	Job queue, rate limits. Holds nothing that can't be lost
E2B	Runs learner and reference code in isolation
Gemini	Generates exercise drafts and live hints. Never grades, never selects
`packages/types`	Shared request/response contracts between web and API

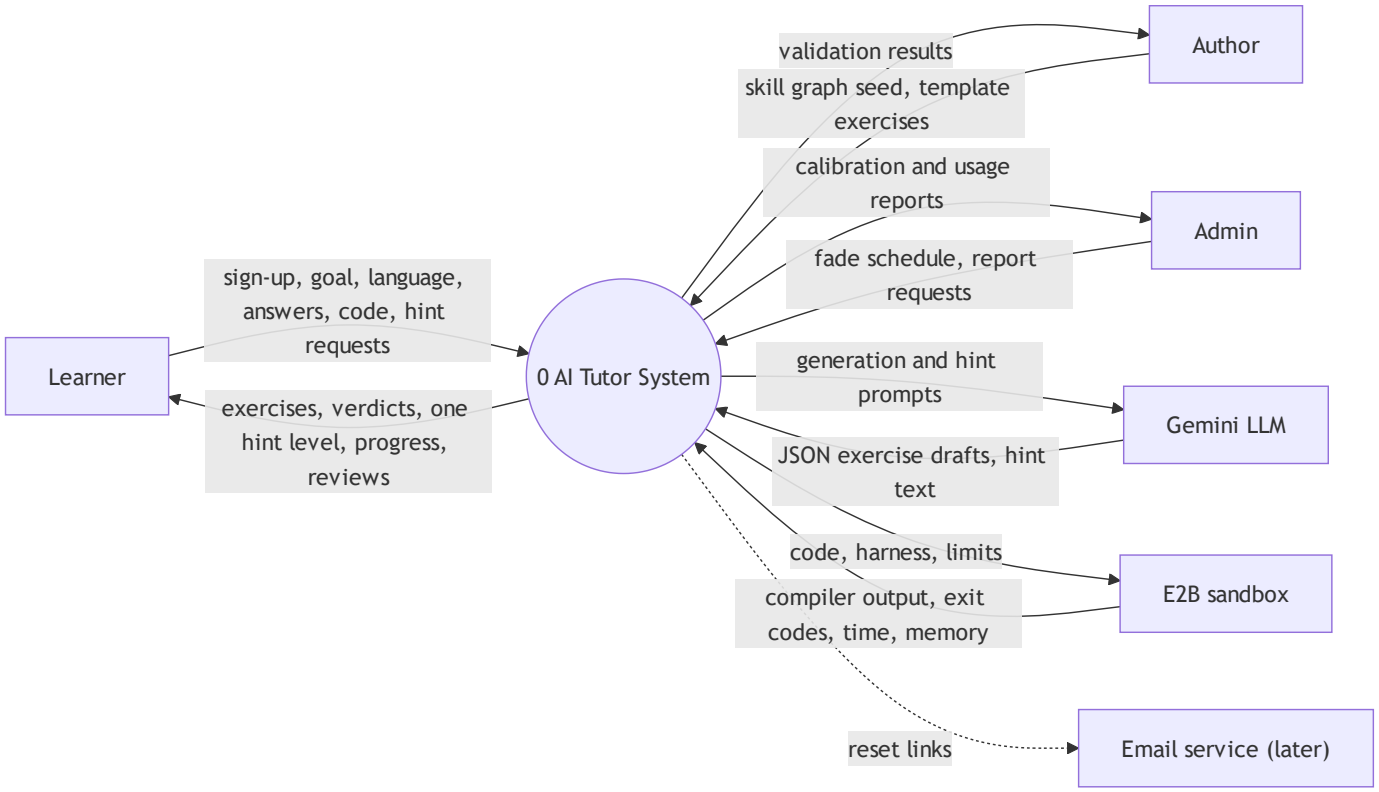
Rule: the API answers fast; anything slow (running code, generating exercises) becomes a queued job handled by the worker.

---

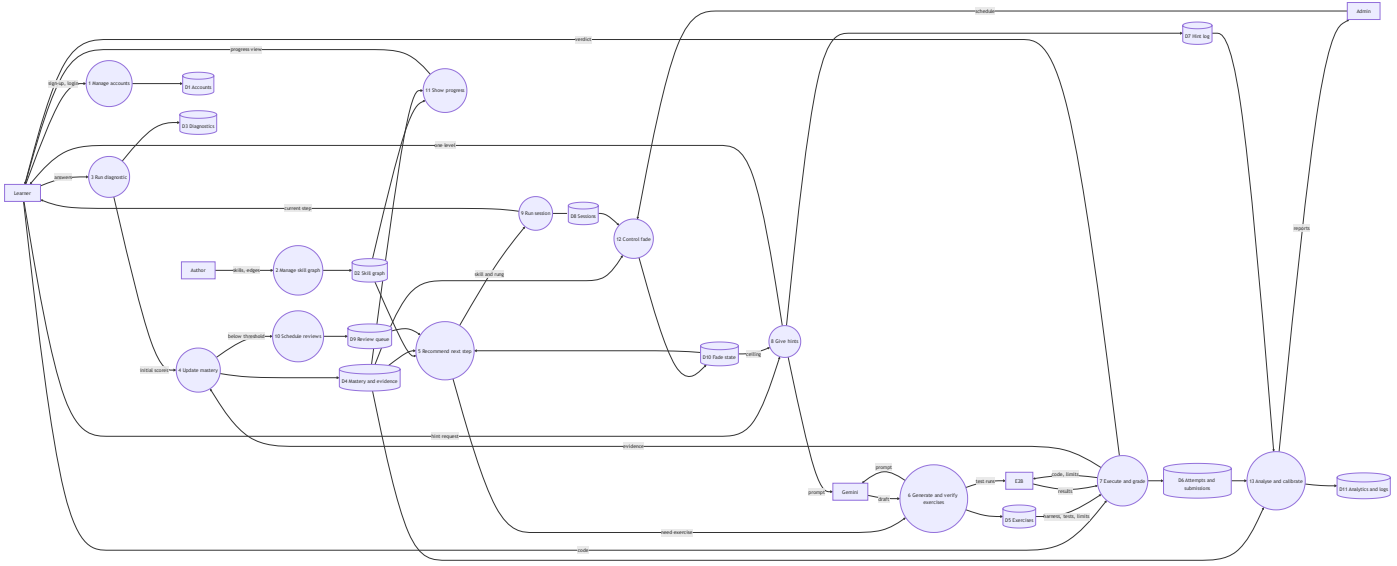
4. Data flow diagrams

Notation: rectangle = external entity · circle = process · cylinder = data store · arrow = data in motion.

4.1 Level 0: context diagram



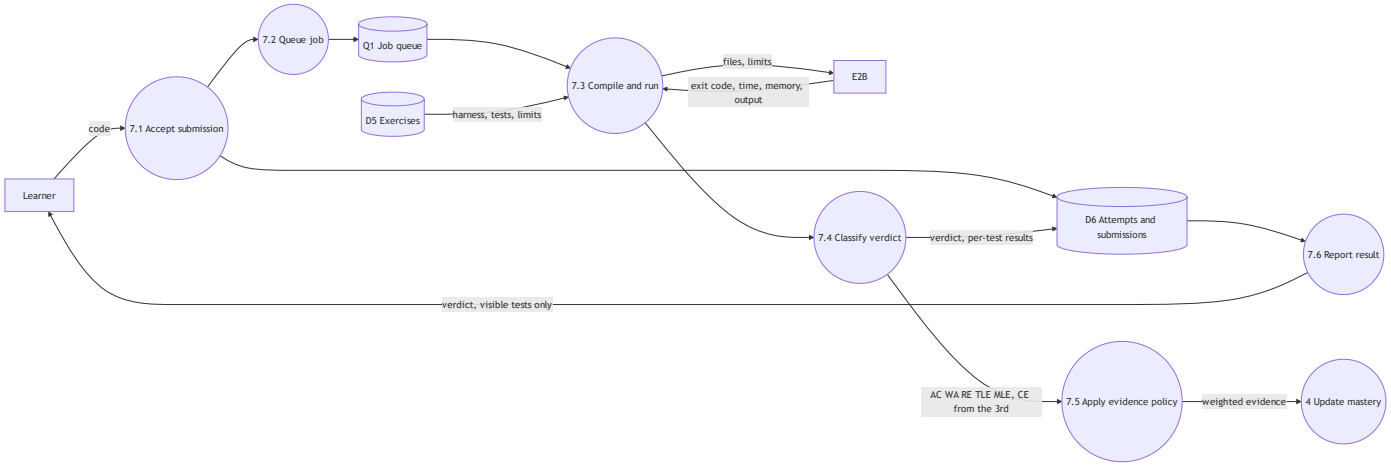
4.2 Level 1: the 13 processes



What the diagram guarantees:

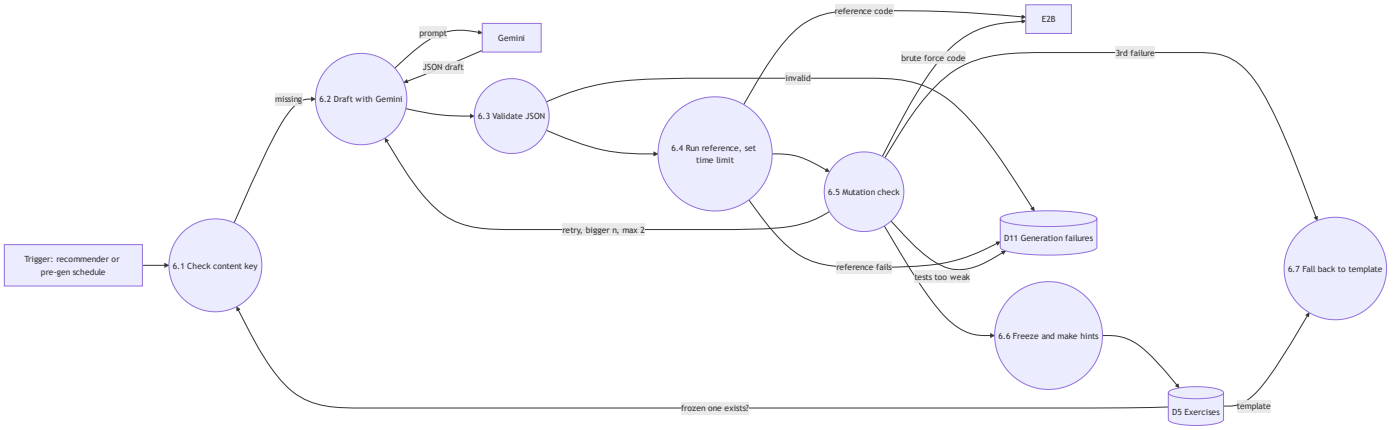
- **Only process 4 writes mastery (D4).** Evidence comes only from the diagnostic (3) and graded code (7). Reading or watching never changes mastery (FR-M4-02).
- **Process 5 has no flow to Gemini.** The next step is chosen by rules (FR-M5-04).
- **Process 8 reads the ceiling from D10** before giving any hint (FR-M12-02).

4.3 Level 2: process 7, Execute and grade



- The harness and hidden tests go to the sandbox only, never to the learner.
- A sandbox error (SE) stops at 7.4 and sends no evidence (NFR-R2).

4.4 Level 2: process 6, Generate and verify exercises



- **Time limit** = 3 × the slowest of several reference runs, never below a measured floor.
- **Mutation check:** a deliberately slow brute-force solution must **pass the small tests** and **time out on the large tests**. This proves the tests can tell an efficient solution from a slow one (FR-M7-03).
- After 3 failed tries the learner gets a hand-written template exercise, with no error shown (NFR-R1).

---

5. Database design and ER diagrams

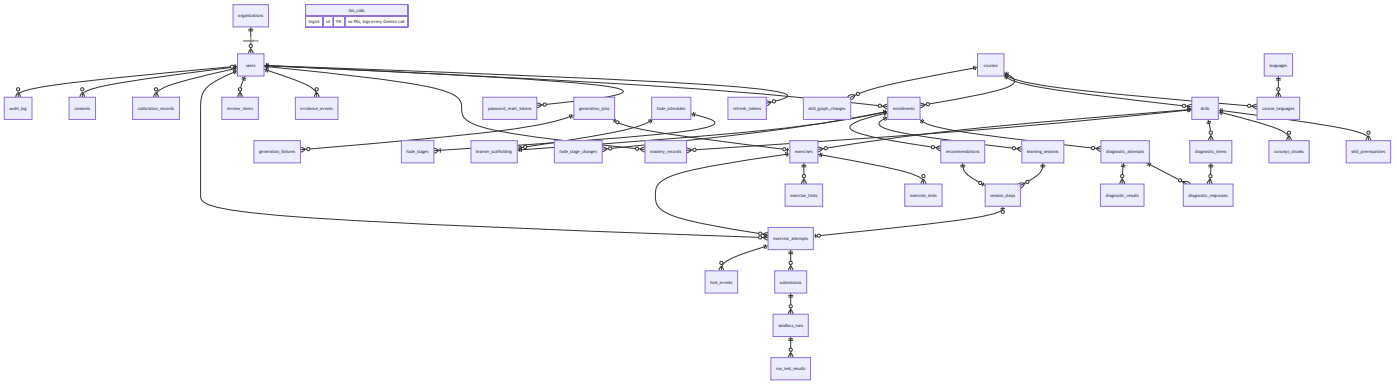
5.1 Design rules

Rule	Meaning	Why
Ledger + snapshot	Every mastery change appends a row to `evidence_events` (never edited); `mastery_records` holds current values; both written in one transaction	Full history for the history API, calibration and research
Frozen content	A verified exercise is never edited; a change creates a new row with a new `content_version`	Determinism
Server-owned state	Hint level, hint ceiling, mastery and verdicts exist only in the database	The client can't tamper with them (spec §23)
Compute on read	Effective mastery = `score × e^(−λ × days since evidence)` is calculated, not stored	It changes every second

Rule	Meaning	Why
Idempotent writes	Unique keys stop a retried job double-counting evidence and a double-click submitting twice	Correctness under retries
Scoped rows	Every user has an `org_id`; every content and mastery row has a `language_code`	Institutions and multiple languages without later migrations

Conventions: UUID primary keys (bigint for high-volume logs), `timestamptz` for all times, enums for fixed value sets, CHECK constraints for ranges (scores 0–1, rung 1–5, hint level 0–5).

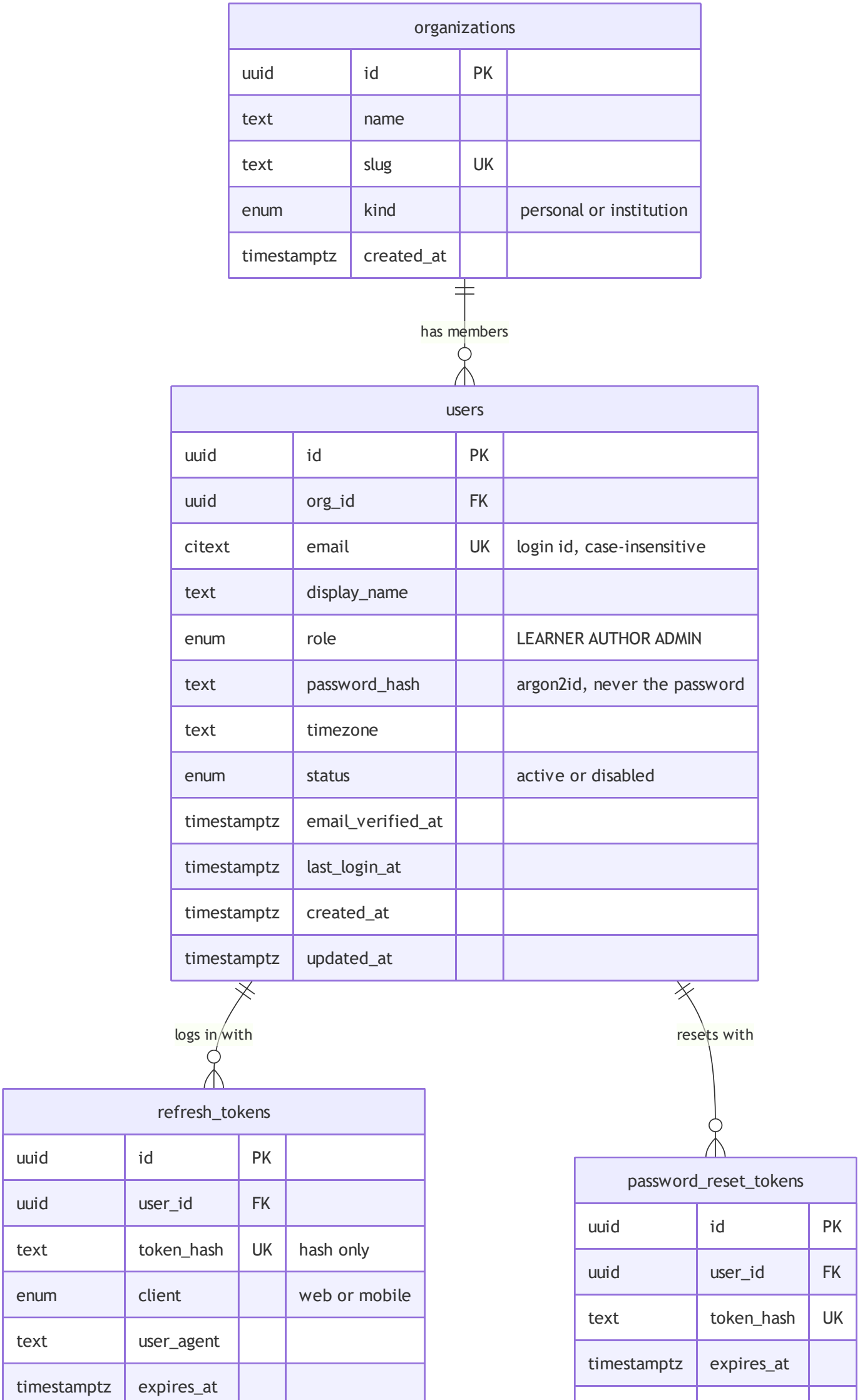
5.2 Overview: 40 tables in 9 groups



Group	Tables	Modules	Built
Identity and access	organizations, users, refresh_tokens, password_reset_tokens	M1	Phase 2 (first 3), later
Courses and skill graph	languages, courses, course_languages, skills, skill_prerequisites, skill_graph_changes, concept_chunks	M2	MVP (first 5), later
Enrollment and diagnostic	enrollments, diagnostic_items, diagnostic_attempts, diagnostic_responses, diagnostic_results	M1, M3	MVP

Group	Tables	Modules	Built
Mastery and reviews	mastery_records, evidence_events, review_items	M4, M10	MVP
Exercises and generation	exercises, exercise_tests, exercise_hints, generation_jobs, generation_failures	M6	MVP
Attempts and execution	exercise_attempts, submissions, sandbox_runs, run_test_results	M7	MVP
Hints, sessions, recommendations	hint_events, learning_sessions, session_steps, recommendations	M5, M8, M9	MVP
Scaffolding fade	fade_schedules, fade_stages, learner_scaffolding, fade_stage_changes	M12	MVP
Analytics, research, cost	calibration_records, consents, llm_calls, audit_log	M13	MVP (first 3), later

5.3 Identity and access (M1)

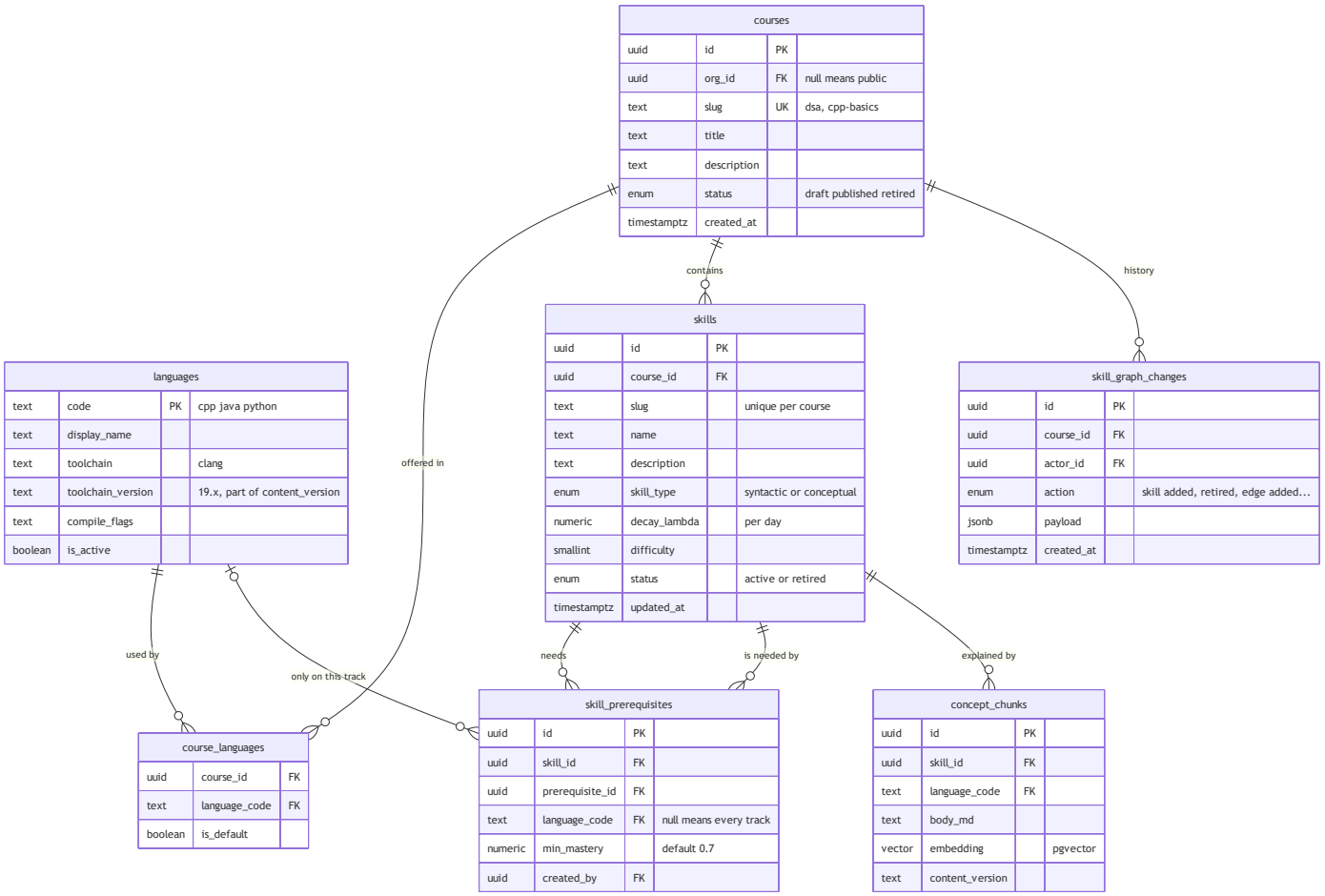

organizations
uuid id PK
text name
text slug UK
enum kind personal or institution
timestamptz created_at
has members
users
uuid id PK
uuid org_id FK
citext email UK login id, case-insensitive
text display_name
enum role LEARNER AUTHOR ADMIN
text password_hash argon2id, never the password
text timezone
enum status active or disabled
timestamptz email_verified_at
timestamptz last_login_at
timestamptz created_at
timestamptz updated_at
logs in with
resets with
refresh_tokens
uuid id PK
uuid user_id FK
text token_hash UK hash only
enum client web or mobile
text user_agent
timestamptz expires_at
password_reset_tokens
uuid id PK
uuid user_id FK
text token_hash UK
timestamptz expires_at

timestamptz	revoked_at		set on logout
timestamptz	created_at		

timestamptz	used_at		

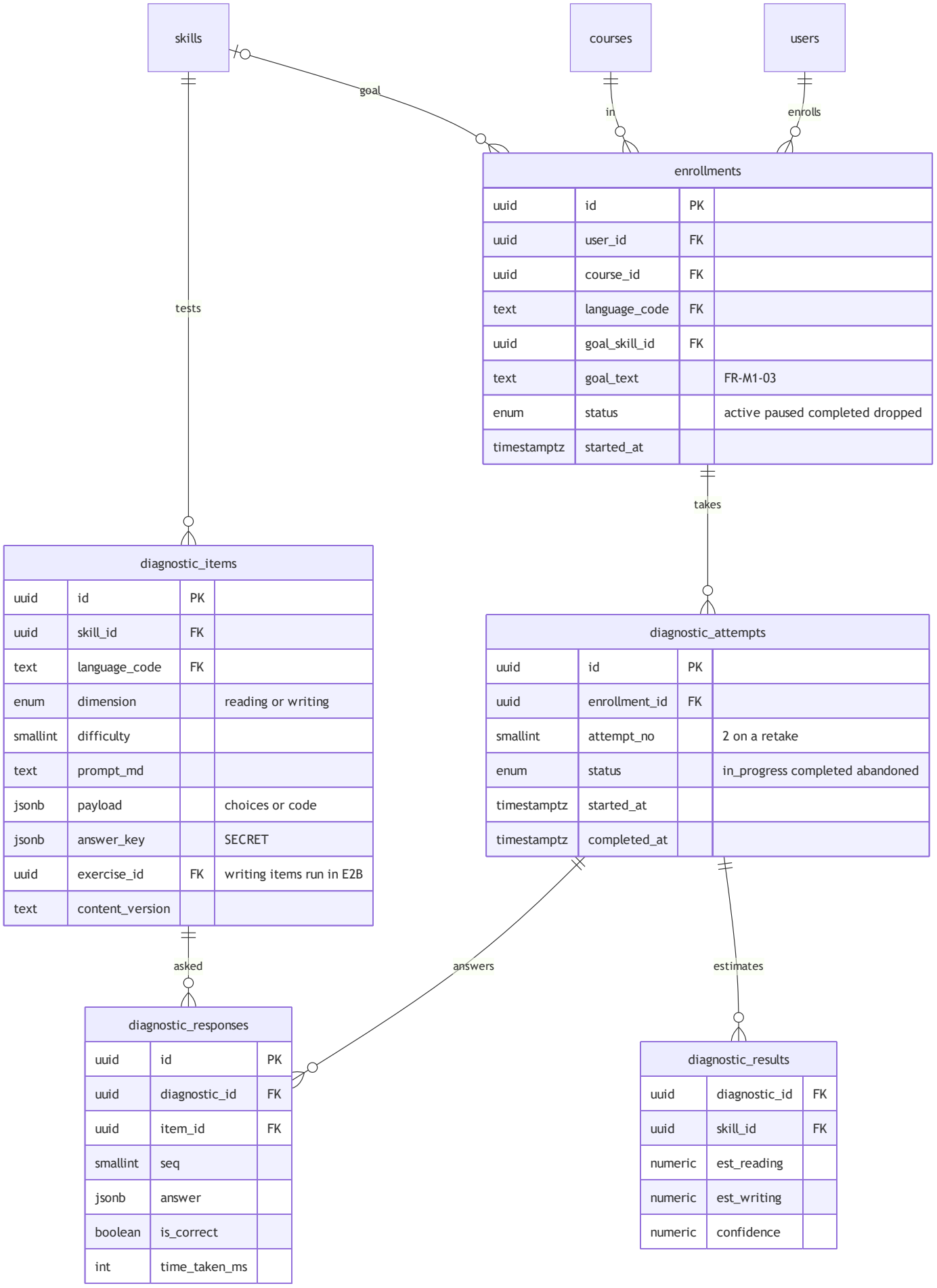
- Login issues a 15-minute JWT (not stored) and a 30-day refresh token (stored only as a hash). Logout sets `revoked_at`.
- Every user belongs to an organisation: solo learners get a personal one; a college becomes an institution organisation.

5.4 Courses and skill graph (M2)



- A **course** is a subject (DSA); a **language** is how it is practised. A beginner C++ track is its own course whose skills DSA skills can depend on.
- Prerequisite edges can apply to one language track only (`language_code`) or to all (`null`).
- The graph must stay acyclic (FR-M2-02): the service runs a cycle check before inserting an edge; a CHECK constraint blocks self-loops.

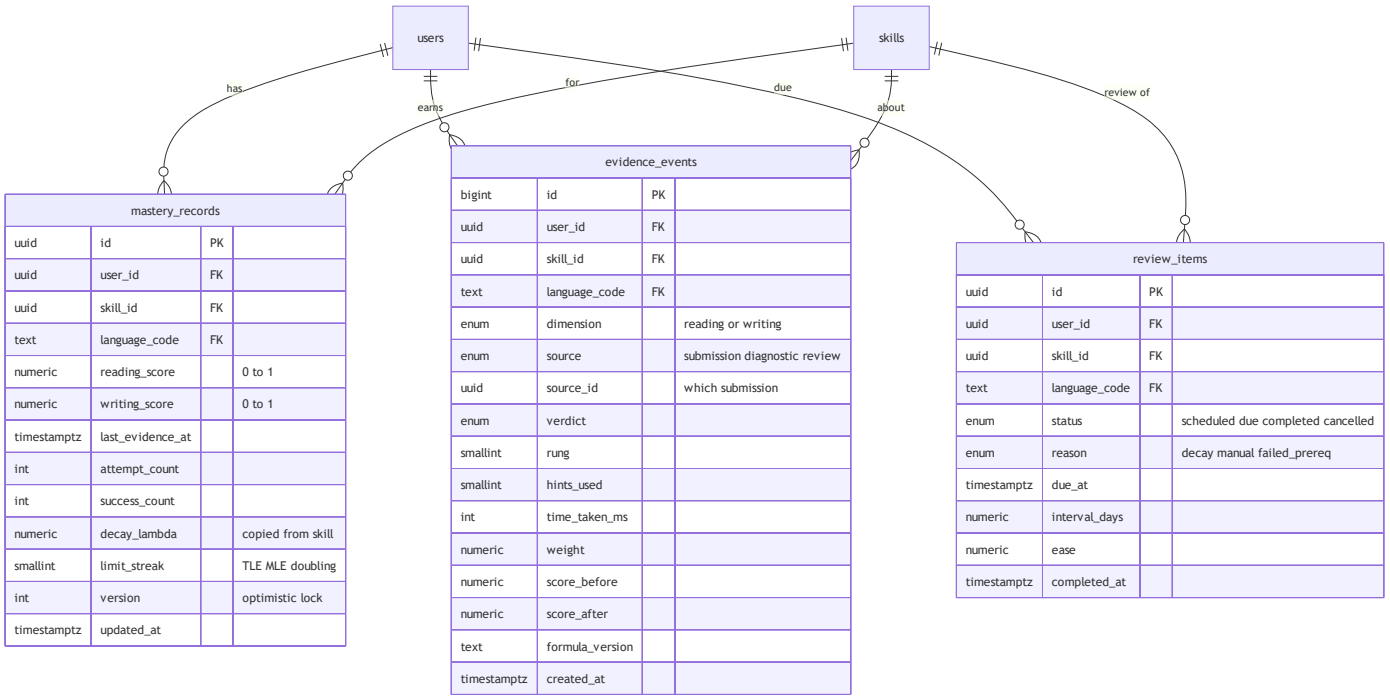
5.5 Enrollment and diagnostic (M1, M3)



- An **enrollment** = user + course + language + goal (one per user, course and language).
- Writing items are real exercises graded in the sandbox, so starting writing scores also come from executed code.

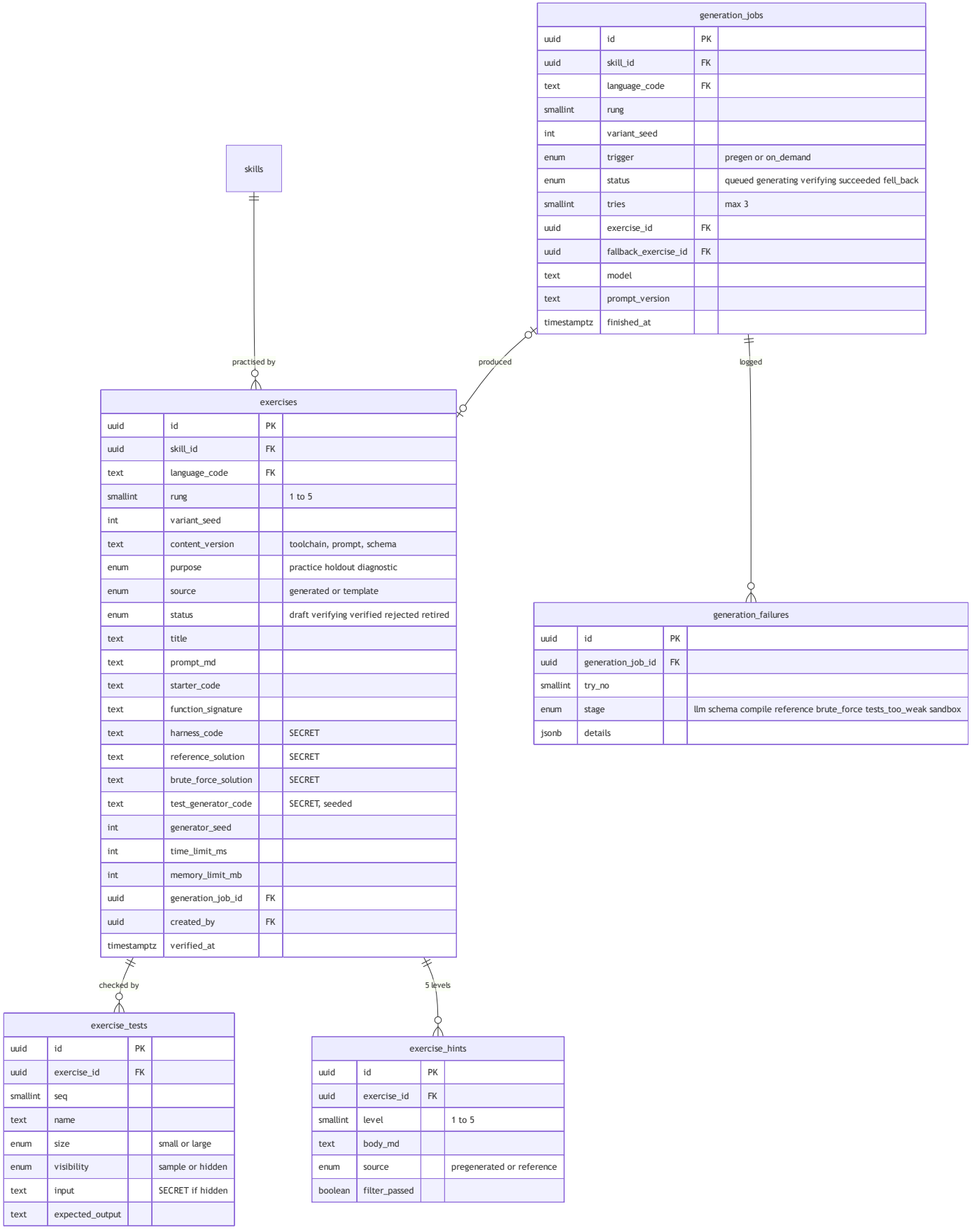
- `diagnostic_results` seeds the first `mastery_records` (FR-M3-02).

5.6 Mastery and reviews (M4, M10)



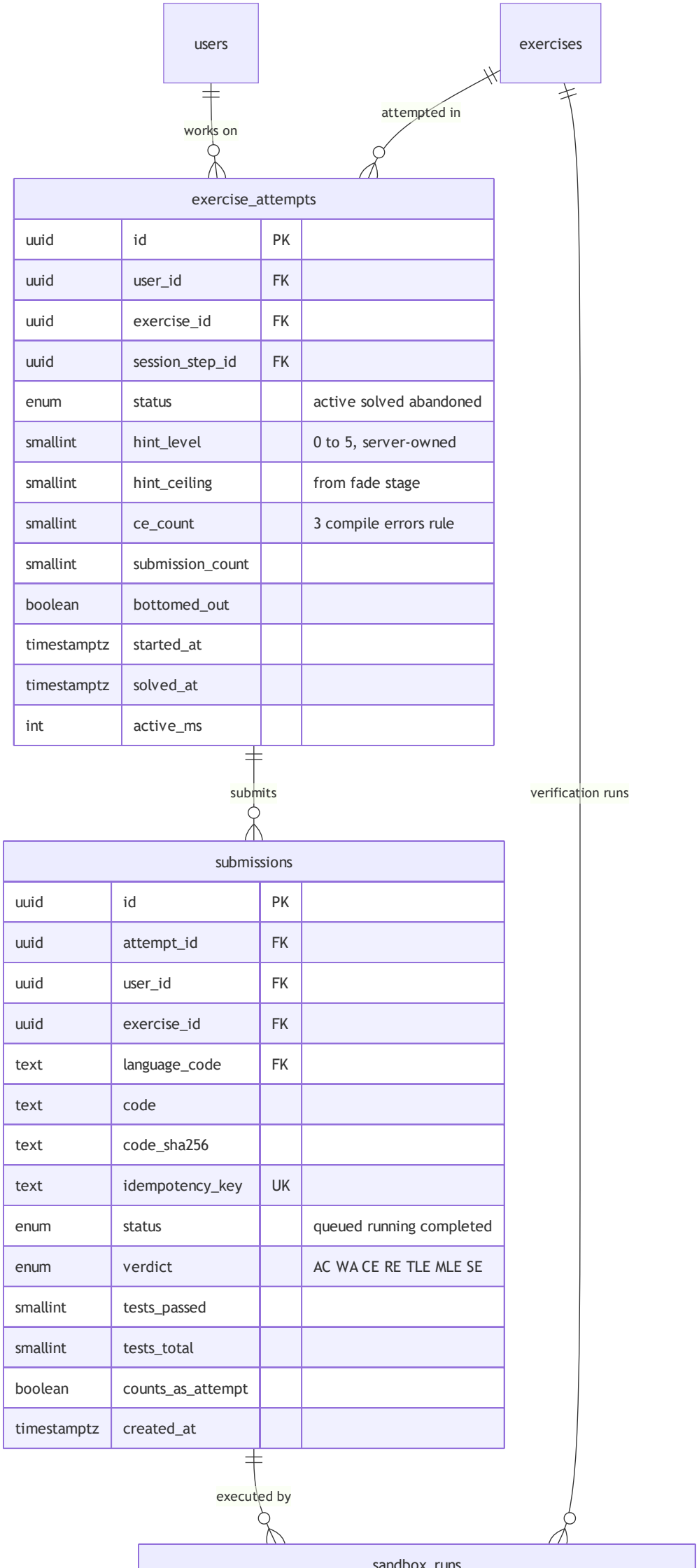
- One mastery row per user, skill and language. Reading and writing are never combined (FR-M11-02).
- `weight` is computed from rung, hints used and time taken (FR-M4-05); `formula_version` keeps old numbers explainable if the formula changes.
- A review is created when effective mastery falls below the threshold (FR-M4-04); only one open review per skill.

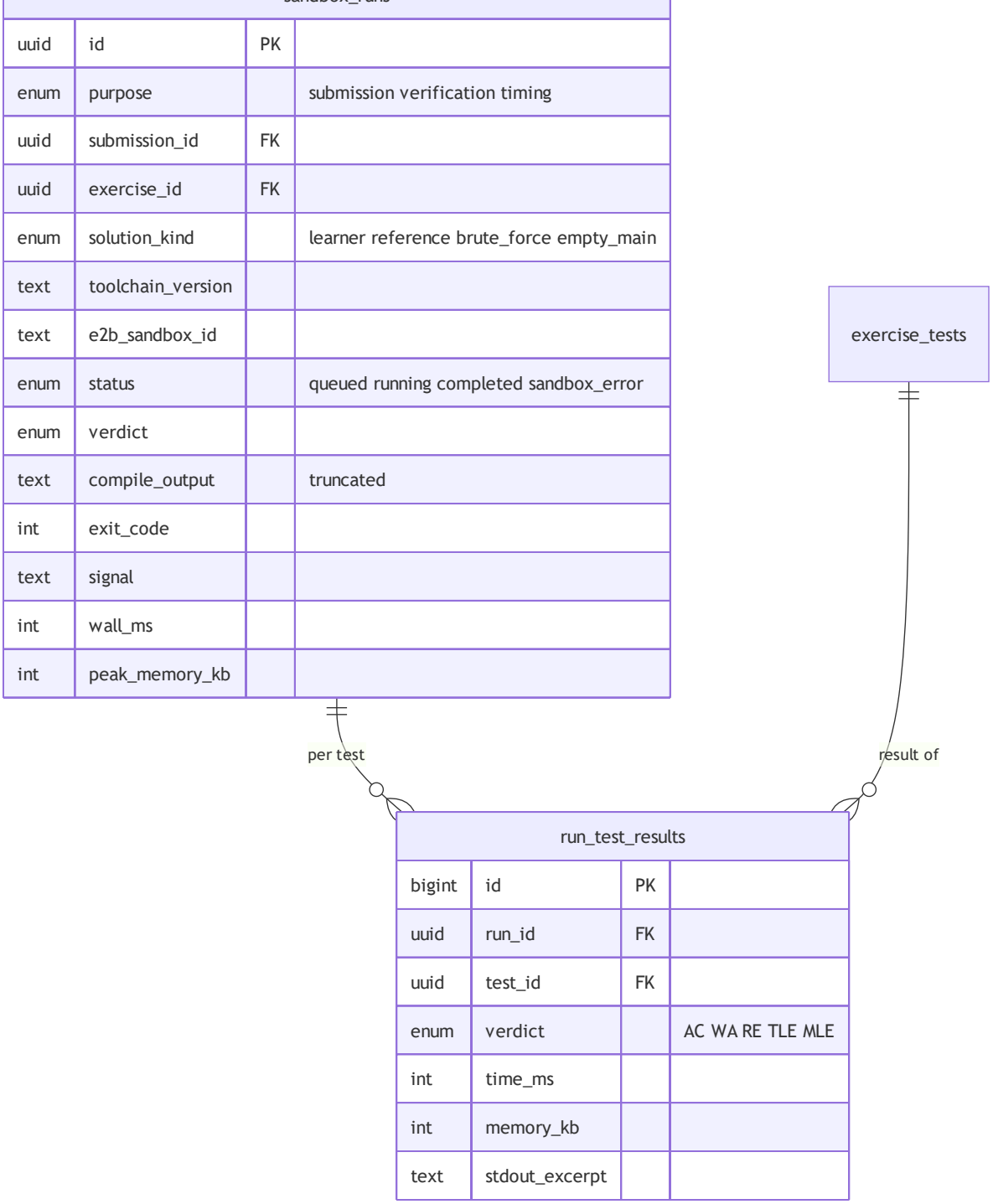
5.7 Exercises and generation (M6)



- **Content key** `(skill, language, rung, variant_seed, content_version)` is unique: the same request always gets the same frozen exercise.
- Columns marked **SECRET** never leave the server.
- Template exercises use the same table (`source = template`) and pass the same verification. `purpose = holdout` exercises are never recommended; they are used only for calibration (M13).

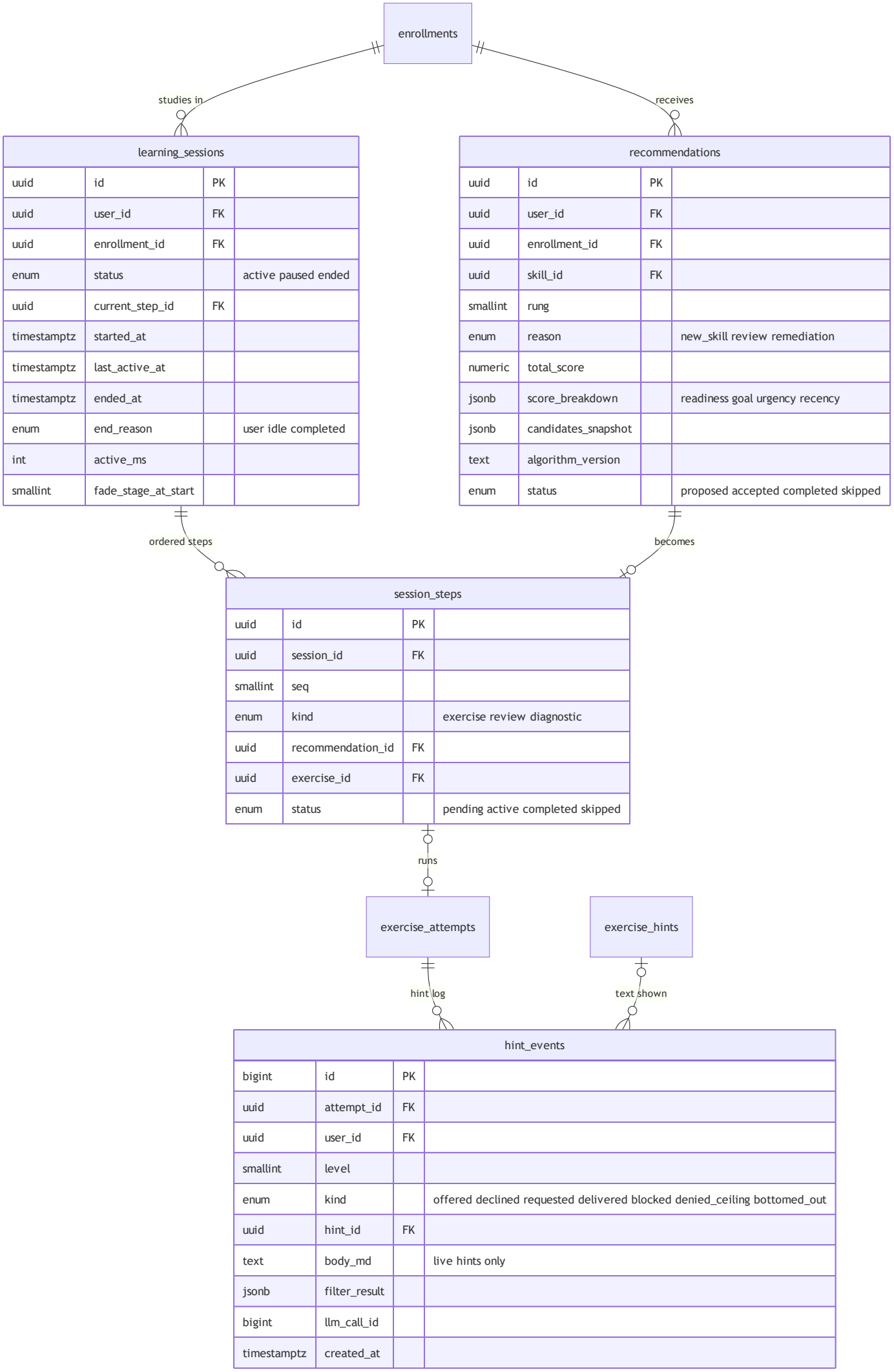
5.8 Attempts and execution (M7)


users
exercises
works on
attempted in
exercise_attempts
uuid id PK
uuid user_id FK
uuid exercise_id FK
uuid session_step_id FK
enum status active solved abandoned
smallint hint_level 0 to 5, server-owned
smallint hint_ceiling from fade stage
smallint ce_count 3 compile errors rule
smallint submission_count
boolean bottomed_out
timestamptz started_at
timestamptz solved_at
int active_ms
submits
verification runs
submissions
uuid id PK
uuid attempt_id FK
uuid user_id FK
uuid exercise_id FK
text language_code FK
text code
text code_sha256
text idempotency_key UK
enum status queued running completed
enum verdict AC WA CE RE TLE MLE SE
smallint tests_passed
smallint tests_total
boolean counts_as_attempt
timestamptz created_at
executed by
sandbox_runs



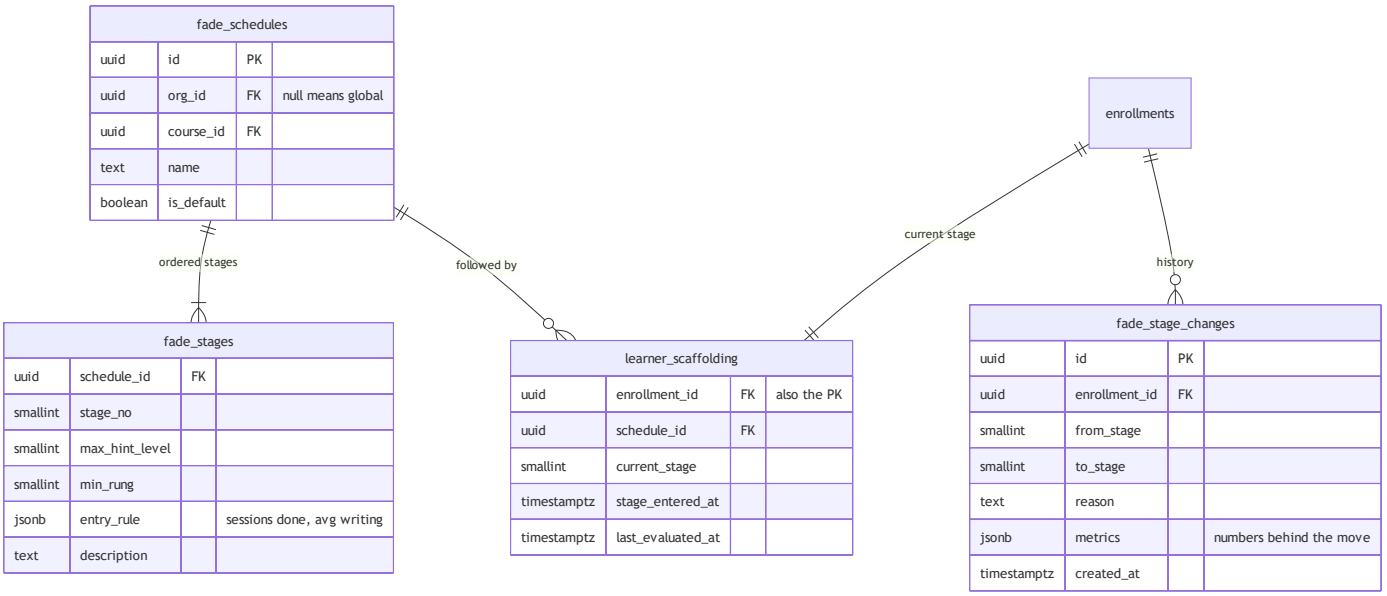
- An **attempt** is one learner on one exercise; it owns the server-side hint level (FR-M8-01). A **submission** is one press of Submit.
- `counts_as_attempt` is false for CE (until the 3rd on the same exercise) and for SE.
- `sandbox_runs` serves both learner submissions and exercise verification, giving one source for execution analytics.

5.9 Hints, sessions and recommendations (M5, M8, M9)


enrollments
studies in
receives
learning_sessions
uuid id PK
uuid user_id FK
uuid enrollment_id FK
enum status active paused ended
uuid current_step_id FK
timestamptz started_at
timestamptz last_active_at
timestamptz ended_at
enum end_reason user idle completed
int active_ms
smallint fade_stage_at_start
recommendations
uuid id PK
uuid user_id FK
uuid enrollment_id FK
uuid skill_id FK
smallint rung
enum reason new_skill review remediation
numeric total_score
jsonb score_breakdown readiness goal urgency recency
jsonb candidates_snapshot
text algorithm_version
enum status proposed accepted completed skipped
ordered steps
becomes
session_steps
uuid id PK
uuid session_id FK
smallint seq
enum kind exercise review diagnostic
uuid recommendation_id FK
uuid exercise_id FK
enum status pending active completed skipped
runs
exercise_attempts
exercise_hints
hint log
text shown
hint_events
bigint id PK
uuid attempt_id FK
uuid user_id FK
smallint level
enum kind offered declined requested delivered blocked denied_ceiling bottomed_out
uuid hint_id FK
text body_md live hints only
jsonb filter_result
bigint llm_call_id
timestamptz created_at

- One active session per learner; a paused session keeps `current_step_id` so it resumes at the same step (FR-M9-03).
- Each recommendation stores its score breakdown and algorithm version, so it can always be explained.

5.10 Scaffolding fade (M12)

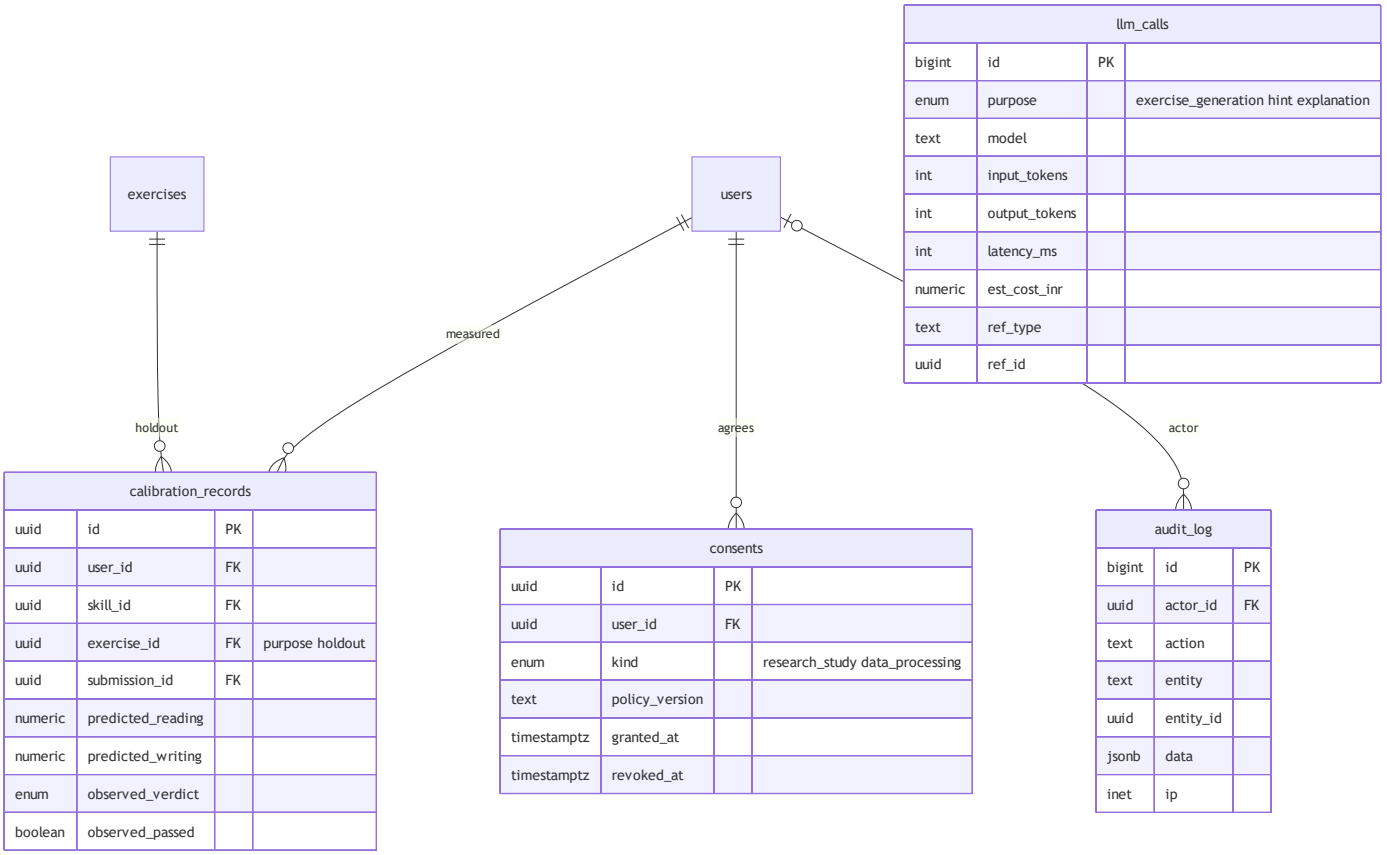


Example schedule (values to be tuned in Phase 3):

Stage	Max hint level	Easiest rung	Enter when
1	5	1	start
2	4	2	4 sessions, average writing ≥ 0.4
3	3	3	8 sessions, average writing ≥ 0.6
4	2	4	12 sessions, average writing ≥ 0.75

The fade is evaluated after each session ends. `fade_stage_changes` records every move, which is the evidence for whether learners need less help over time.

5.11 Analytics, research and cost (M13)



- **Calibration** compares predicted mastery with the result on a held-out exercise (FR-M13-01).
- **Consent** is recorded before any learner data is used in the college study.
- `llm_calls` tracks token use and estimated cost of every Gemini call.
- Hint trends, bottom-out rate, exercise effectiveness, generation failure rate and execution statistics are **SQL views** over existing tables, not extra tables.

5.12 Key constraints

Table	Constraint	Enforces
users	UNIQUE(email)	One account per email (FR-M1-01)
skills	UNIQUE(course_id, slug)	No duplicate skills
skill_prerequisites	CHECK(skill_id ≠ prerequisite_id); one edge per pair per track	No self-loops or duplicate edges
enrollments	UNIQUE(user_id, course_id, language_code)	One enrollment per track
mastery_records	UNIQUE(user_id, skill_id, language_code); CHECK scores 0–1	One current value per skill

Table	Constraint	Enforces
evidence_events	UNIQUE(source, source_id, dimension)	Evidence never counted twice
review_items	Partial UNIQUE: one open review per skill	No duplicate reviews
exercises	UNIQUE content key; CHECK rung 1–5	Determinism
exercise_attempts	Partial UNIQUE(user_id, exercise_id) where active	One live attempt
submissions	UNIQUE(idempotency_key)	No duplicate submissions
learning_sessions	Partial UNIQUE(user_id) where not ended	One active session

Every foreign key that is queried is indexed.

5.13 Build order

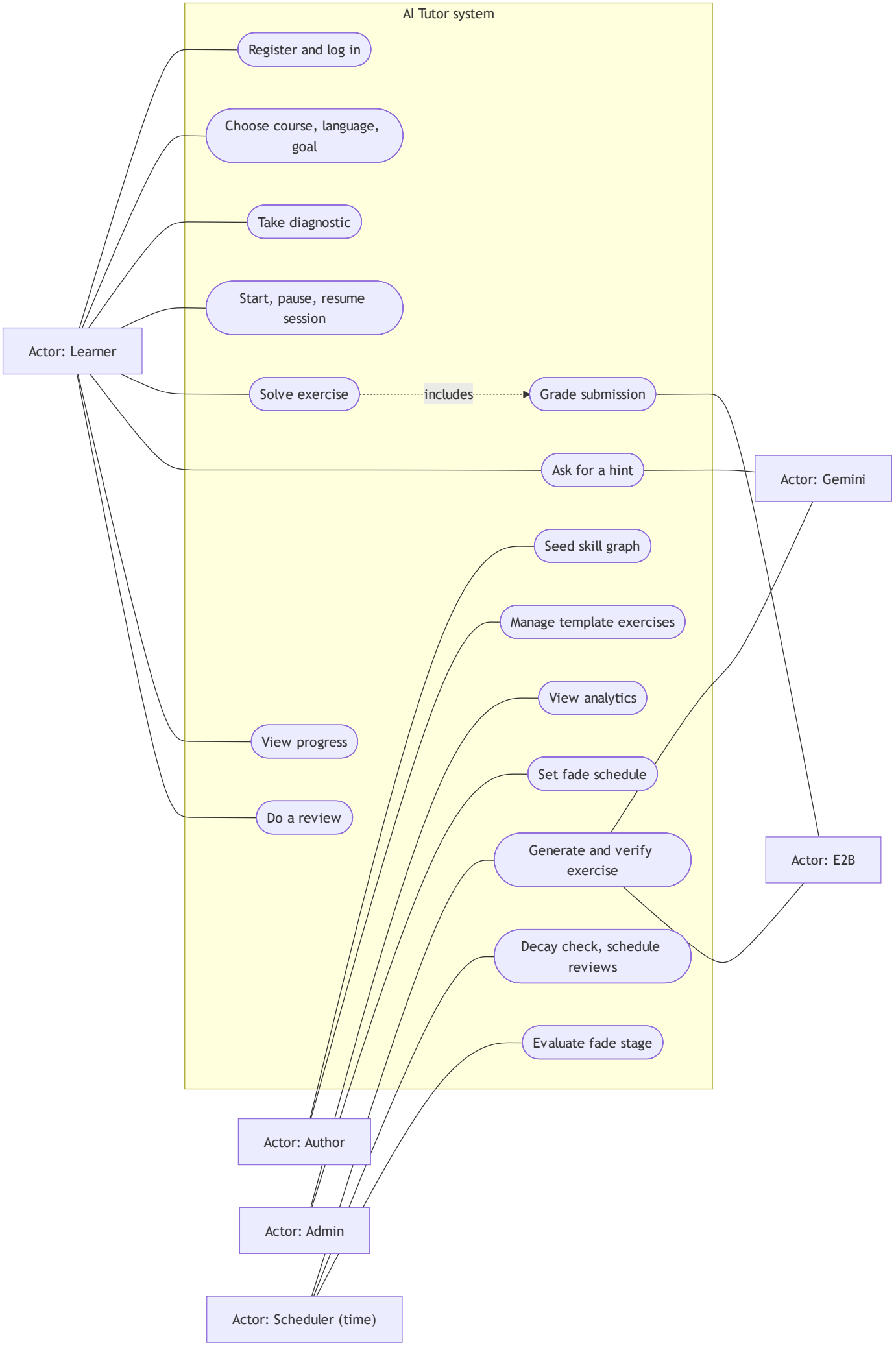
Phase	Tables
Phase 2 (15 Oct)	organizations, users, refresh_tokens
MVP (10 Nov)	33 more, in learning-loop order: skill graph → enrollment and diagnostic → mastery → exercises → attempts and execution → hints → sessions and recommendations → fade → analytics
Later	password_reset_tokens, skill_graph_changes, concept_chunks, audit_log

Each step is a versioned Drizzle migration; applied migrations are never edited.

---

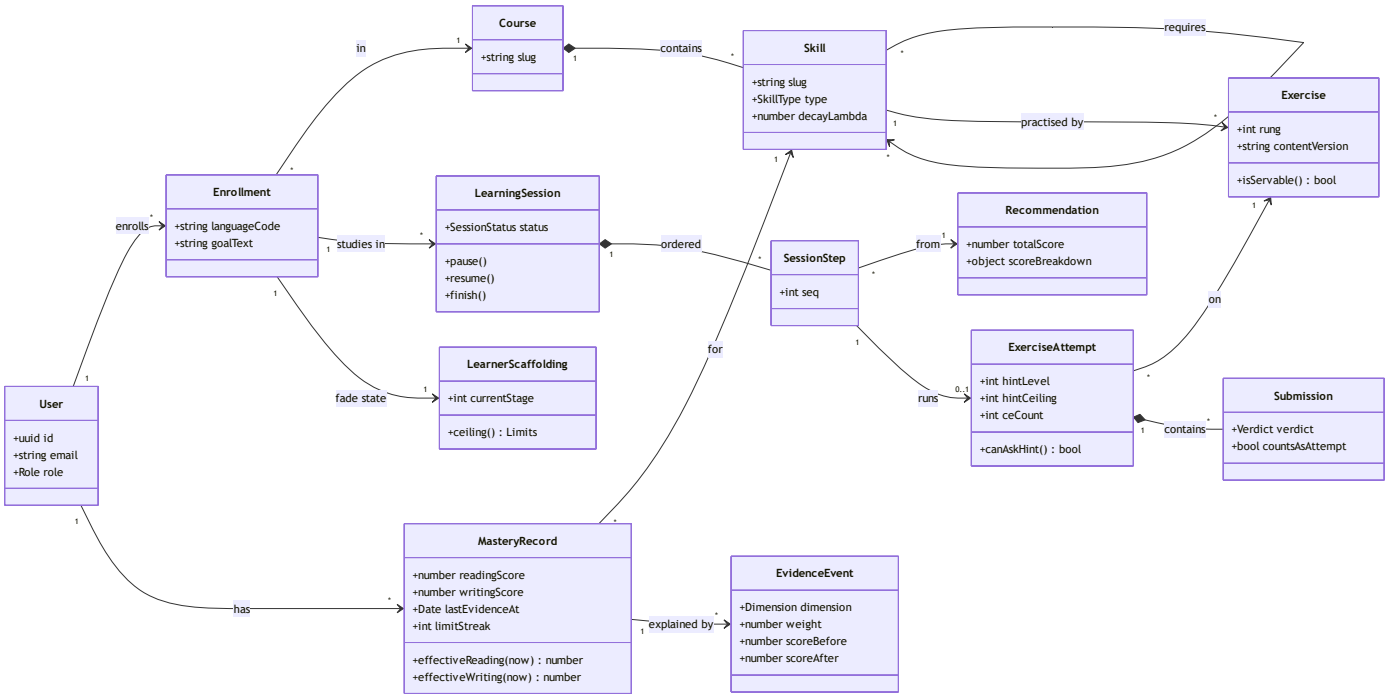
6. UML diagrams

6.1 Use case diagram


AI Tutor system
Register and log in
Choose course, language, goal
Take diagnostic
Start, pause, resume session
Actor: Learner
Solve exercise
includes
Grade submission
Ask for a hint
Actor: Gemini
Seed skill graph
Manage template exercises
View analytics
View progress
Set fade schedule
Do a review
Generate and verify exercise
Actor: E2B
Decay check, schedule reviews
Evaluate fade stage
Actor: Author
Actor: Admin
Actor: Scheduler (time)

The scheduler is an actor because mastery decays and the fade advances with time, not only with clicks.

6.2 Class diagram: domain model



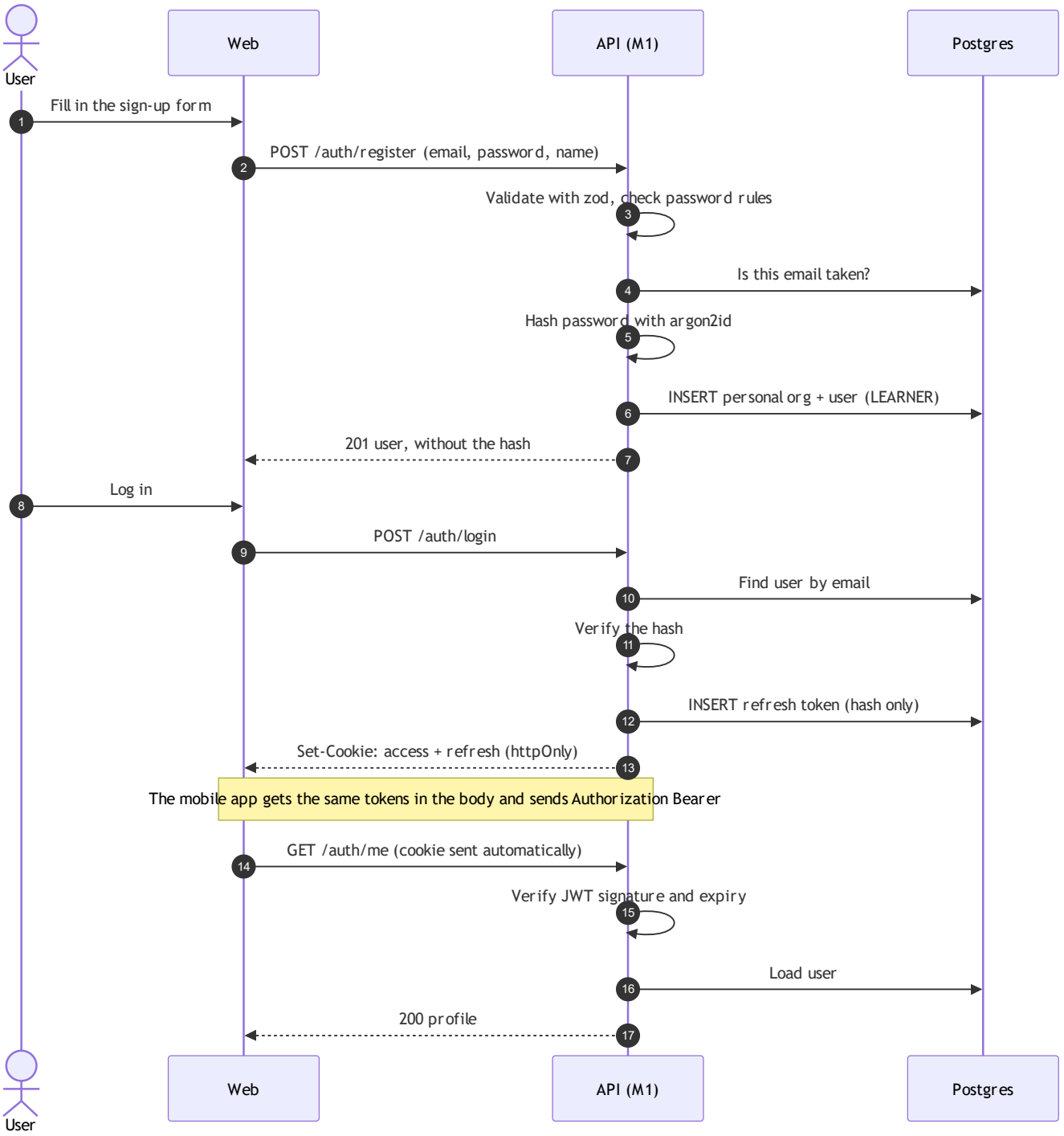
6.3 Class diagram: services and adapters

classDiagram direction TB class JudgeService { +grade(submission) Verdict } class MasteryService { +applyEvidence(evidence) +effective(record, now) number } class RecommenderService { +next(enrollment) Recommendation } class HintEngine { +requestHint(attempt) Hint } class OutputFilter { +isSolutionLike(text, exercise) bool } class FadeController { +ceiling(enrollment) Limits +evaluate(enrollment) } class ExercisePipeline { +generate(contentKey) Exercise } class MasteryRepository { +findForUpdate(user, skill, language) +save(record, evidence) } class SandboxProvider { <> +compile(files, language) CompileResult +run(binary, input, limits) RunResult } class LlmProvider { <> +generateJson(prompt, schema) object +generateText(prompt) string } class E2BSandbox class GeminiLlm E2BSandbox ..|> SandboxProvider GeminiLlm ..|> LlmProvider JudgeService --> SandboxProvider JudgeService --> MasteryService ExercisePipeline --> LlmProvider ExercisePipeline --> SandboxProvider HintEngine --> FadeController HintEngine --> OutputFilter HintEngine --> LlmProvider RecommenderService --> MasteryService RecommenderService --> FadeController MasteryService --> MasteryRepository

- External services sit behind interfaces (`SandboxProvider`, `LlmProvider`); replacing E2B or Gemini means writing one new class.

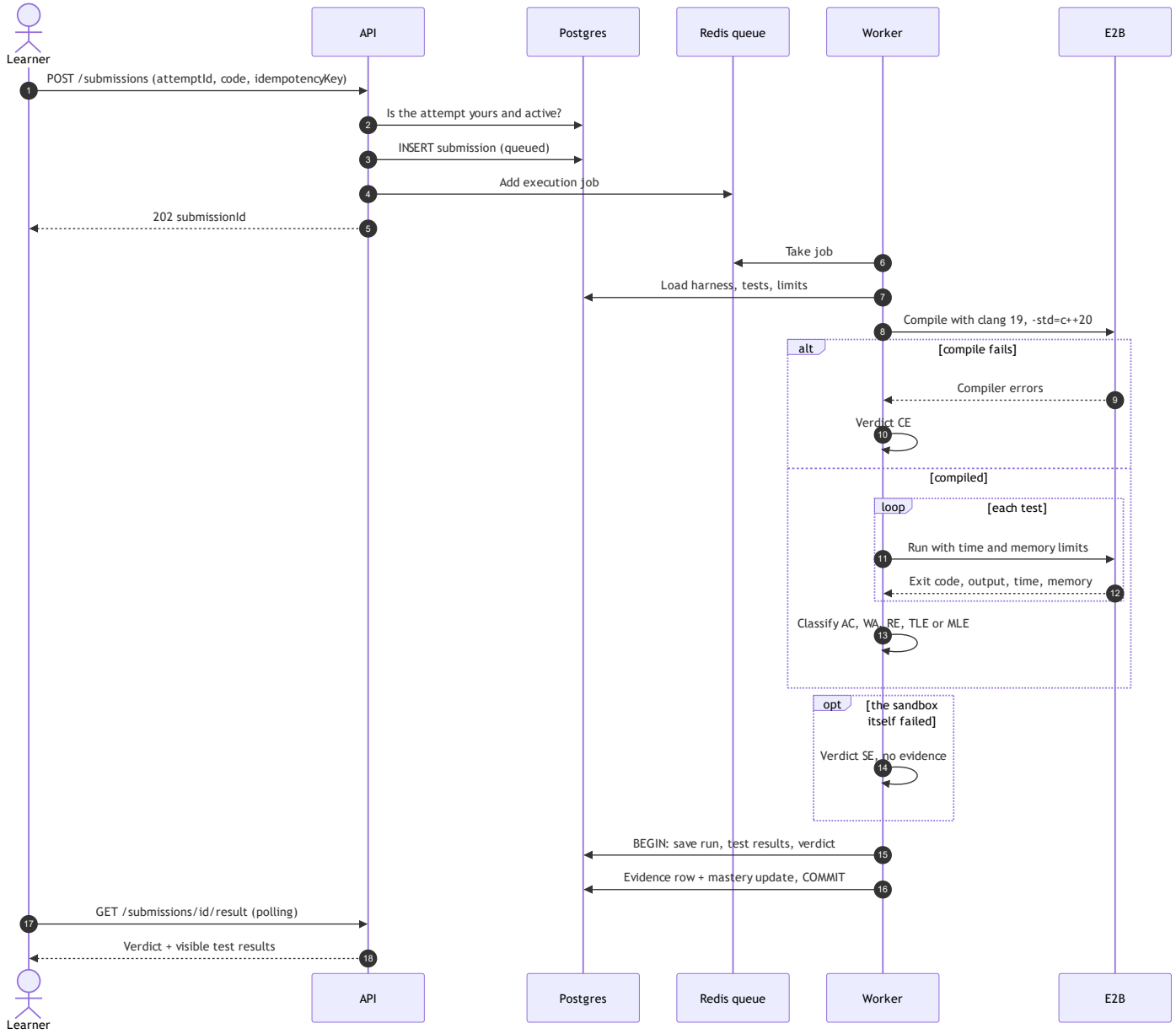
- Layering per request: route → controller (validate, call service) → service (rules) → repository (SQL).

6.4 Sequence: register and log in

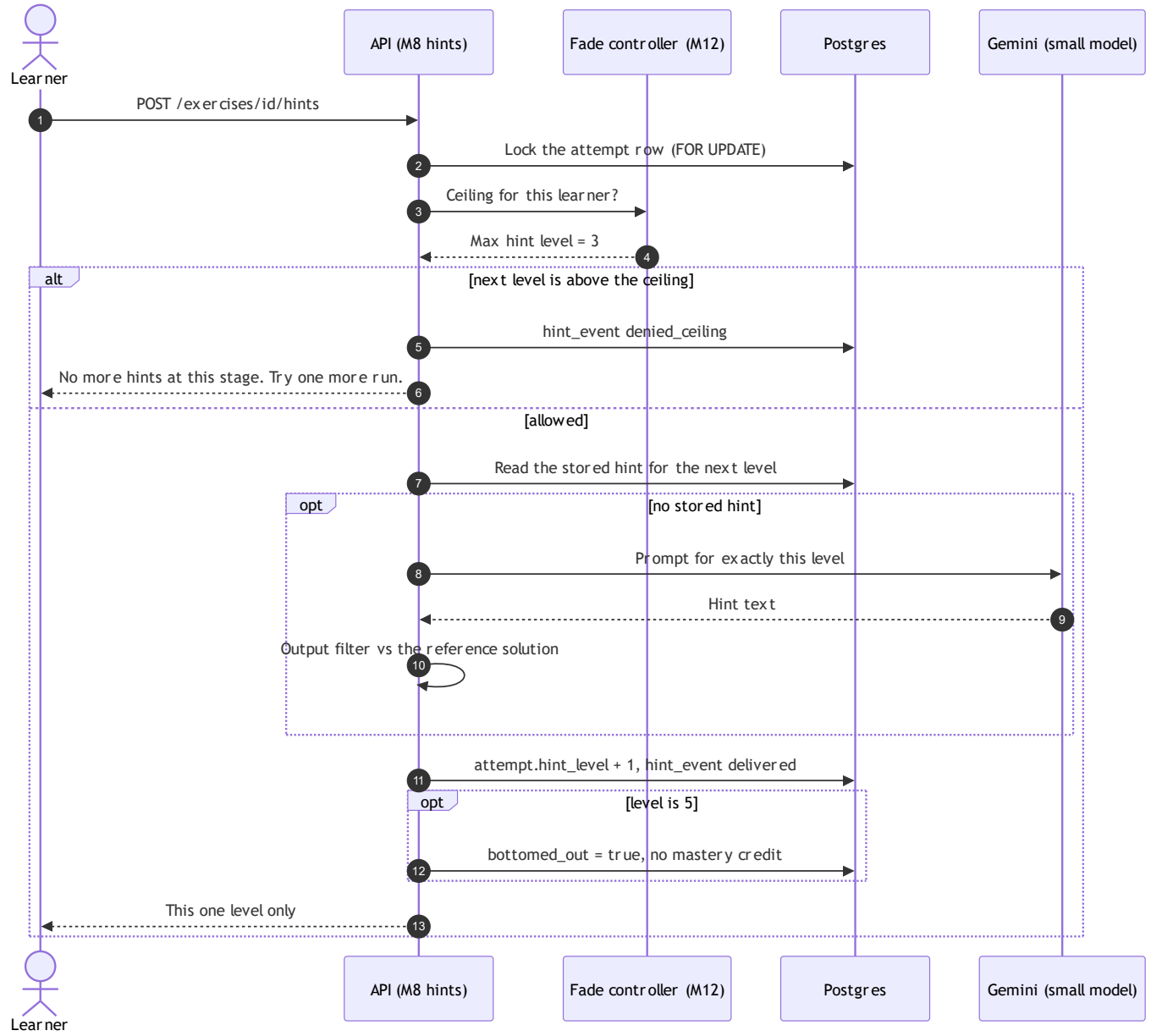


Wrong email and wrong password return the same error; login is rate-limited.

6.5 Sequence: submit code

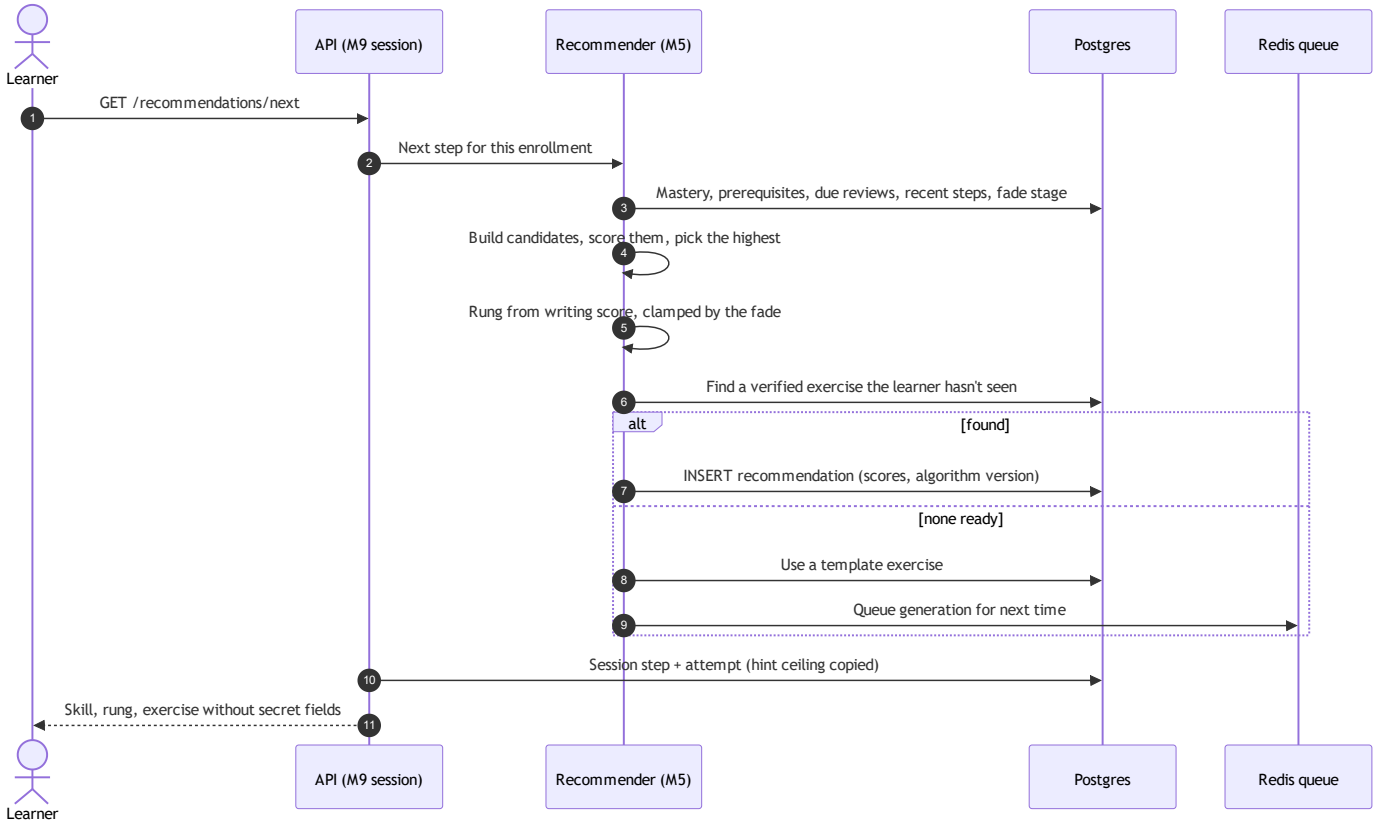


6.6 Sequence: request a hint

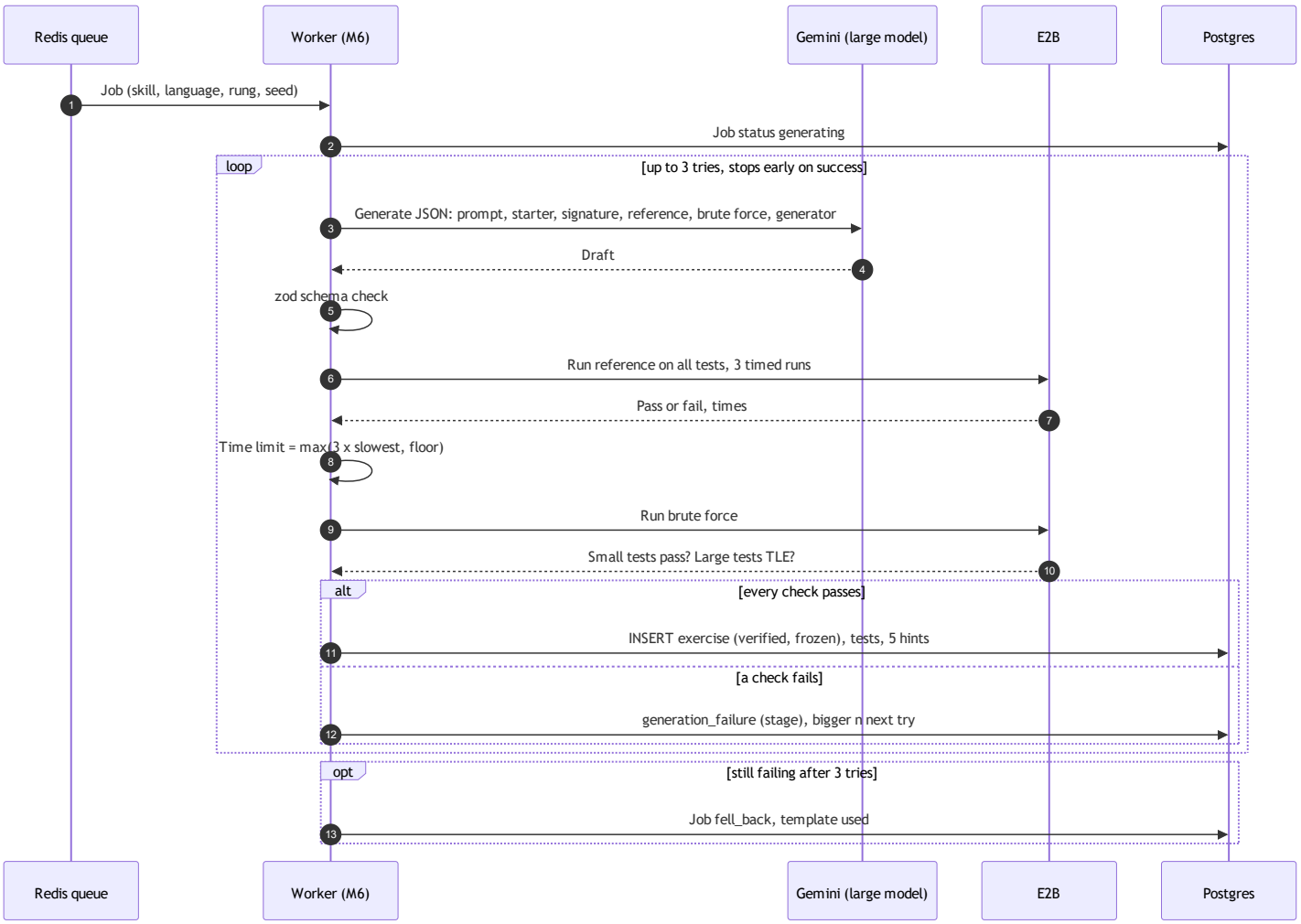


Hint ladder: 1 Concept · 2 Location · 3 Mechanism · 4 Worked example · 5 Answer (frozen reference solution).

6.7 Sequence: get the next step

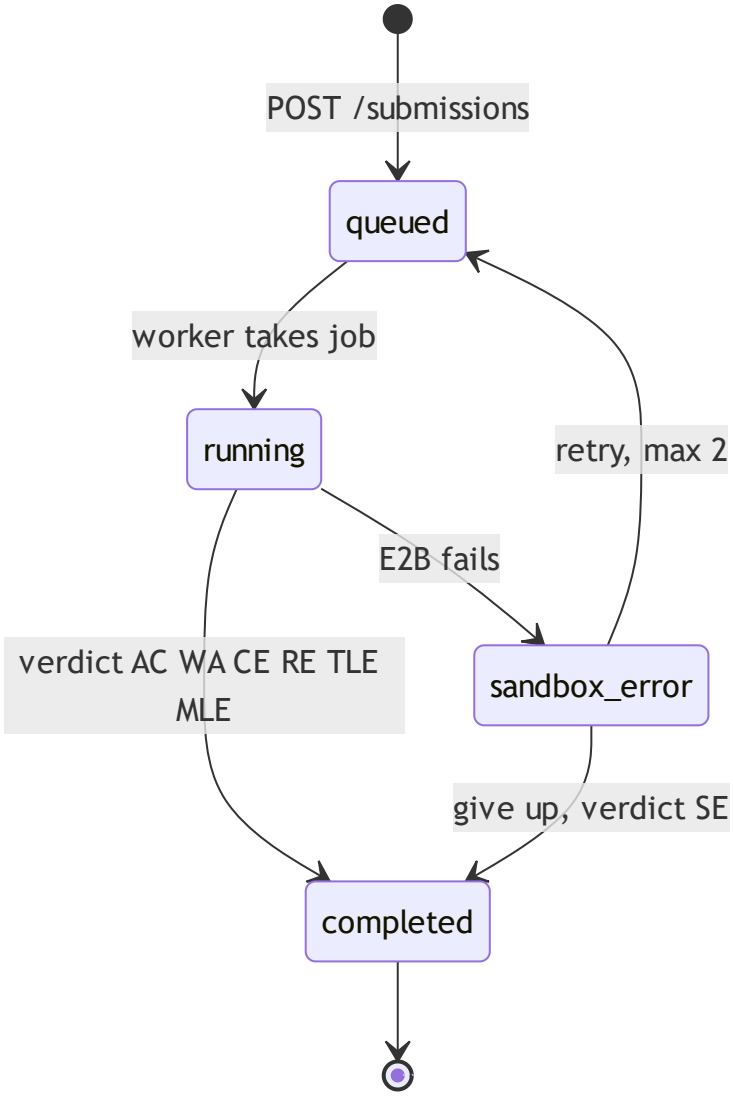


6.8 Sequence: pre-generate an exercise

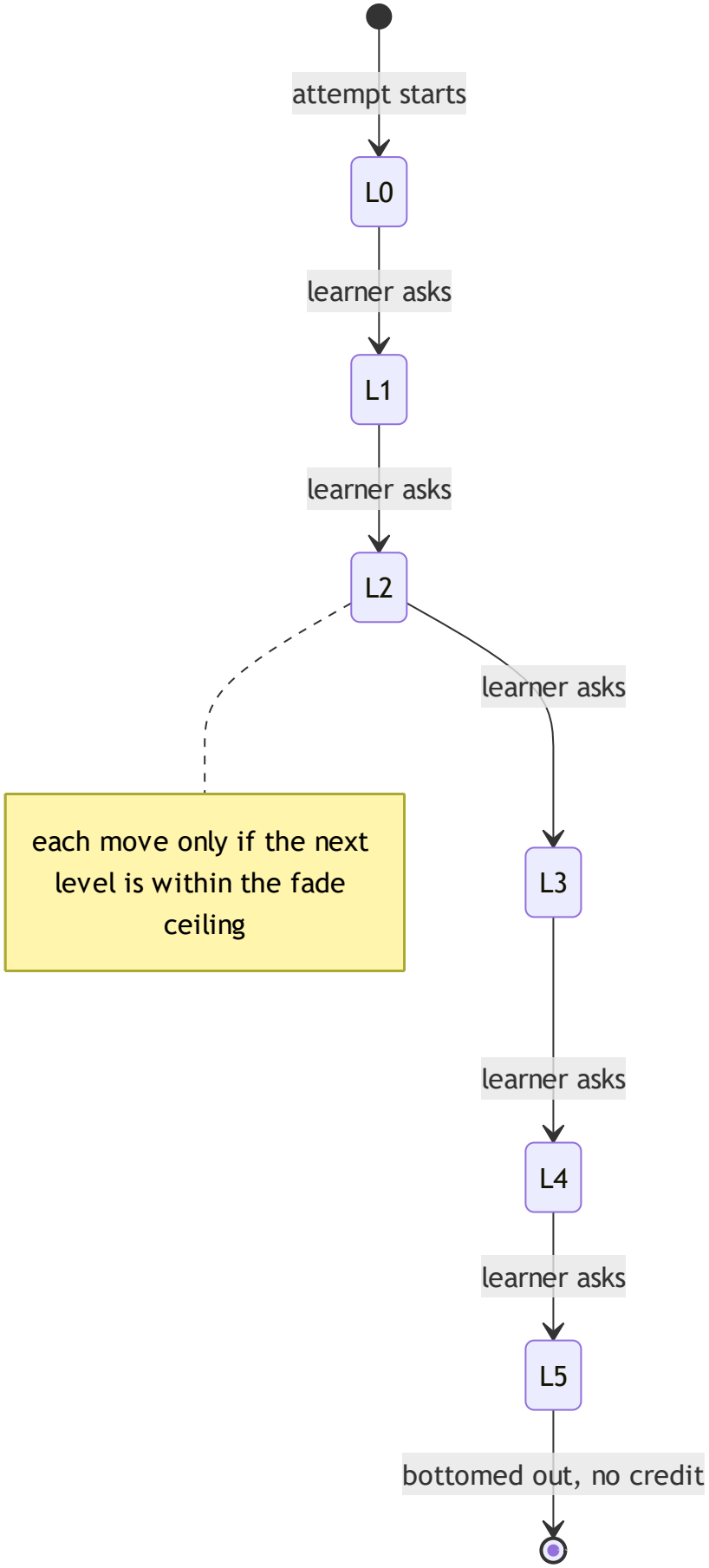


6.9 State diagrams

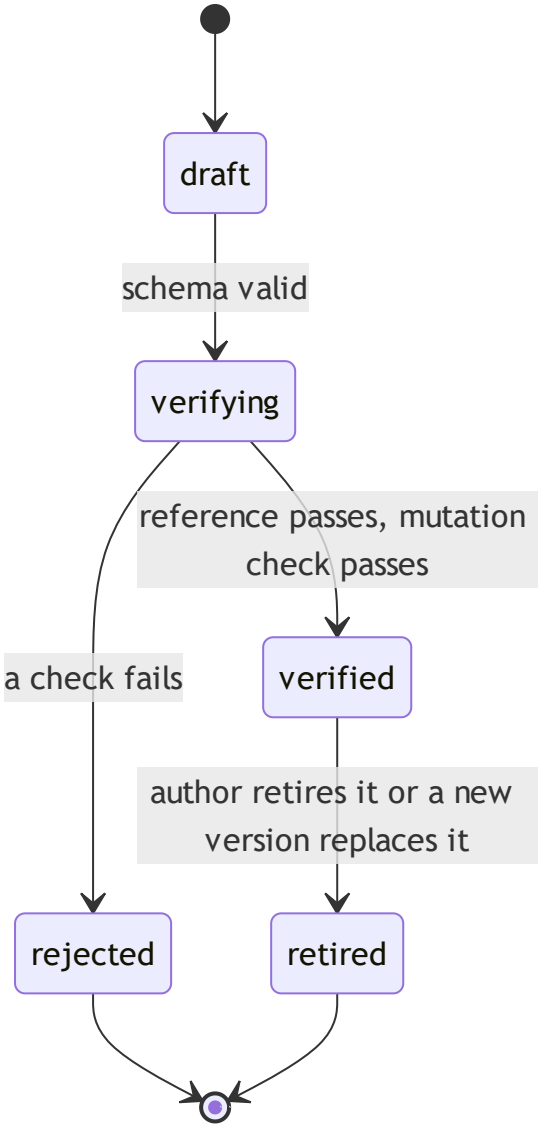
Submission



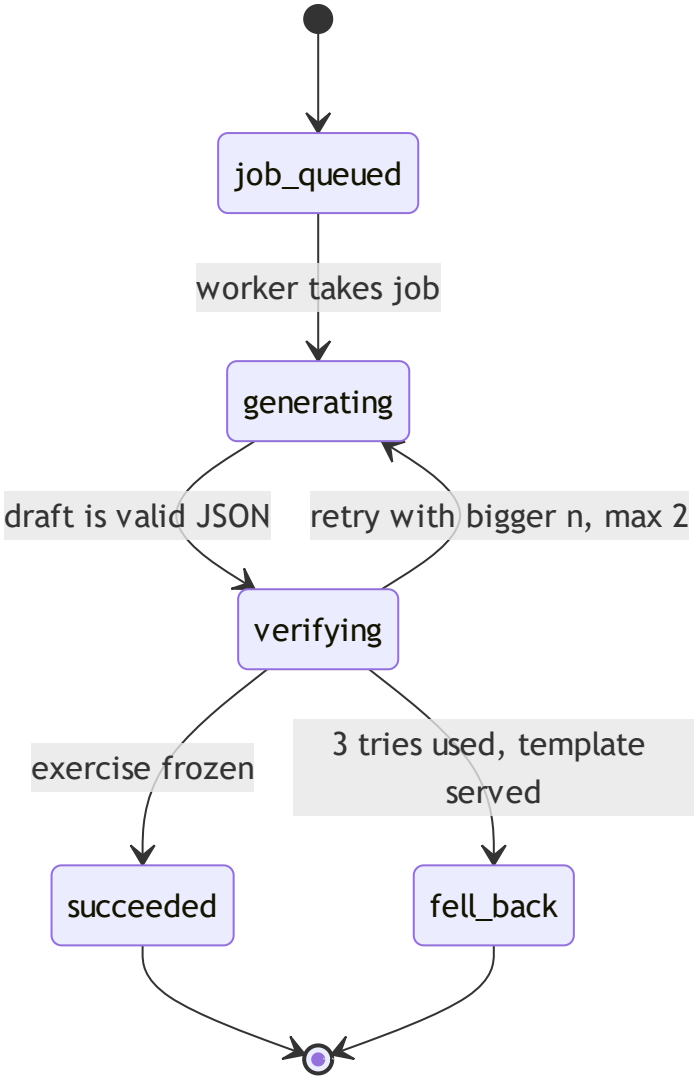
Hint ladder (per attempt)


attempt starts
L0
learner asks
L1
learner asks
L2
learner asks
each move only if the next level is within the fade ceiling
L3
learner asks
L4
learner asks
L5
bottomed out, no credit

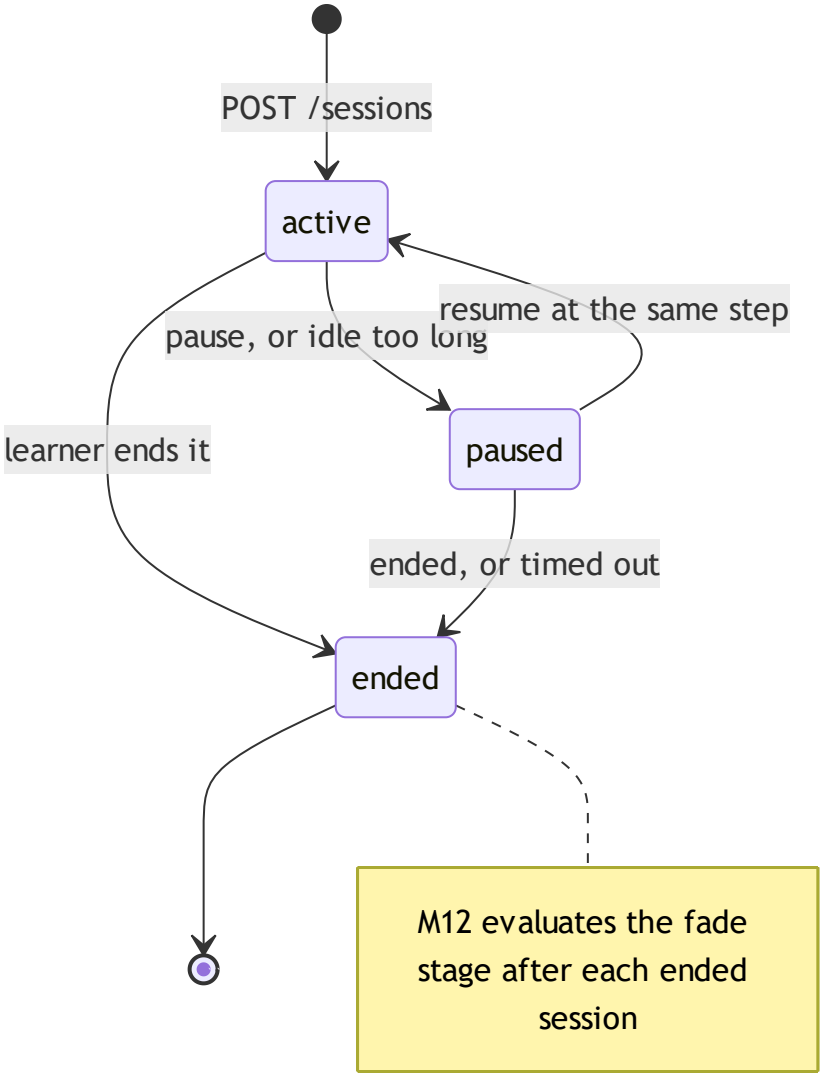
Exercise



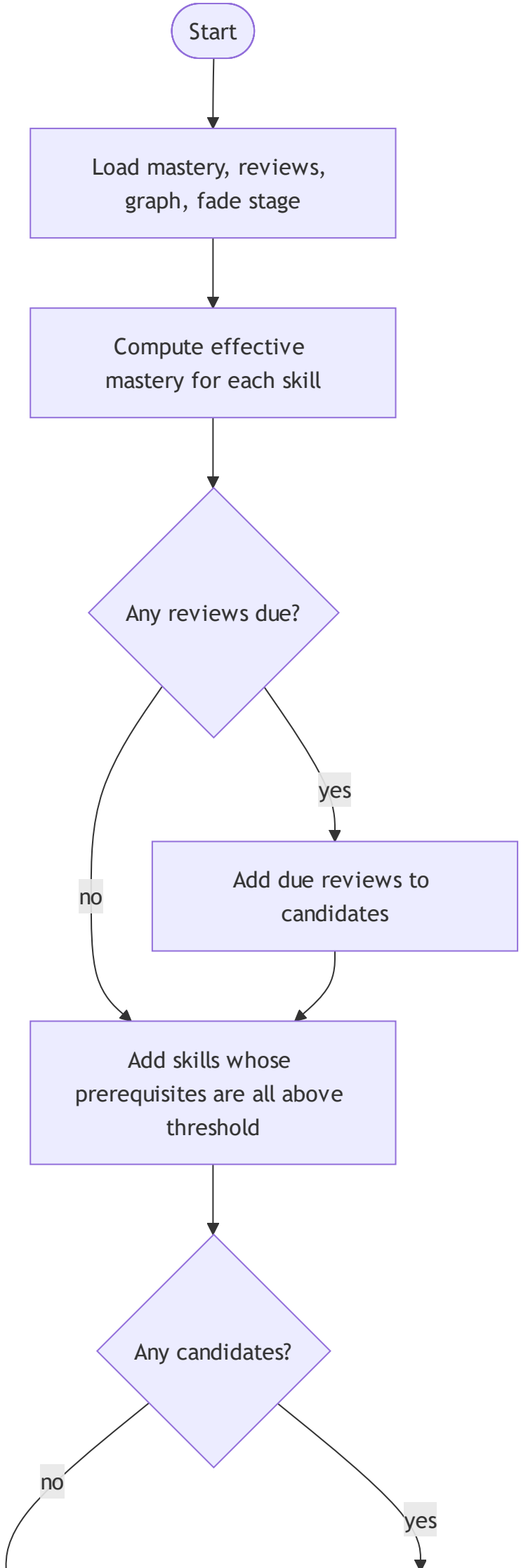
Generation job

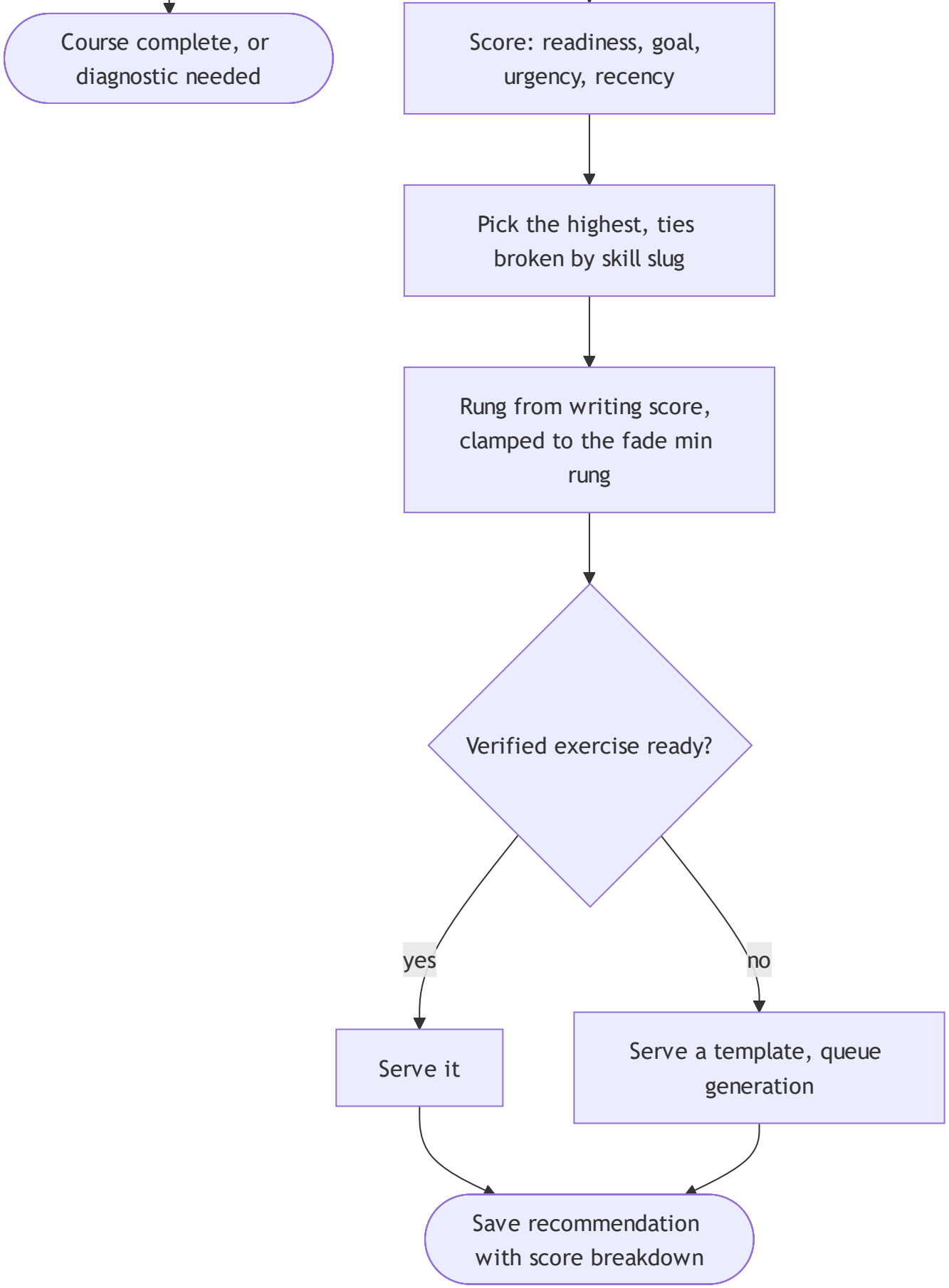


Learning session

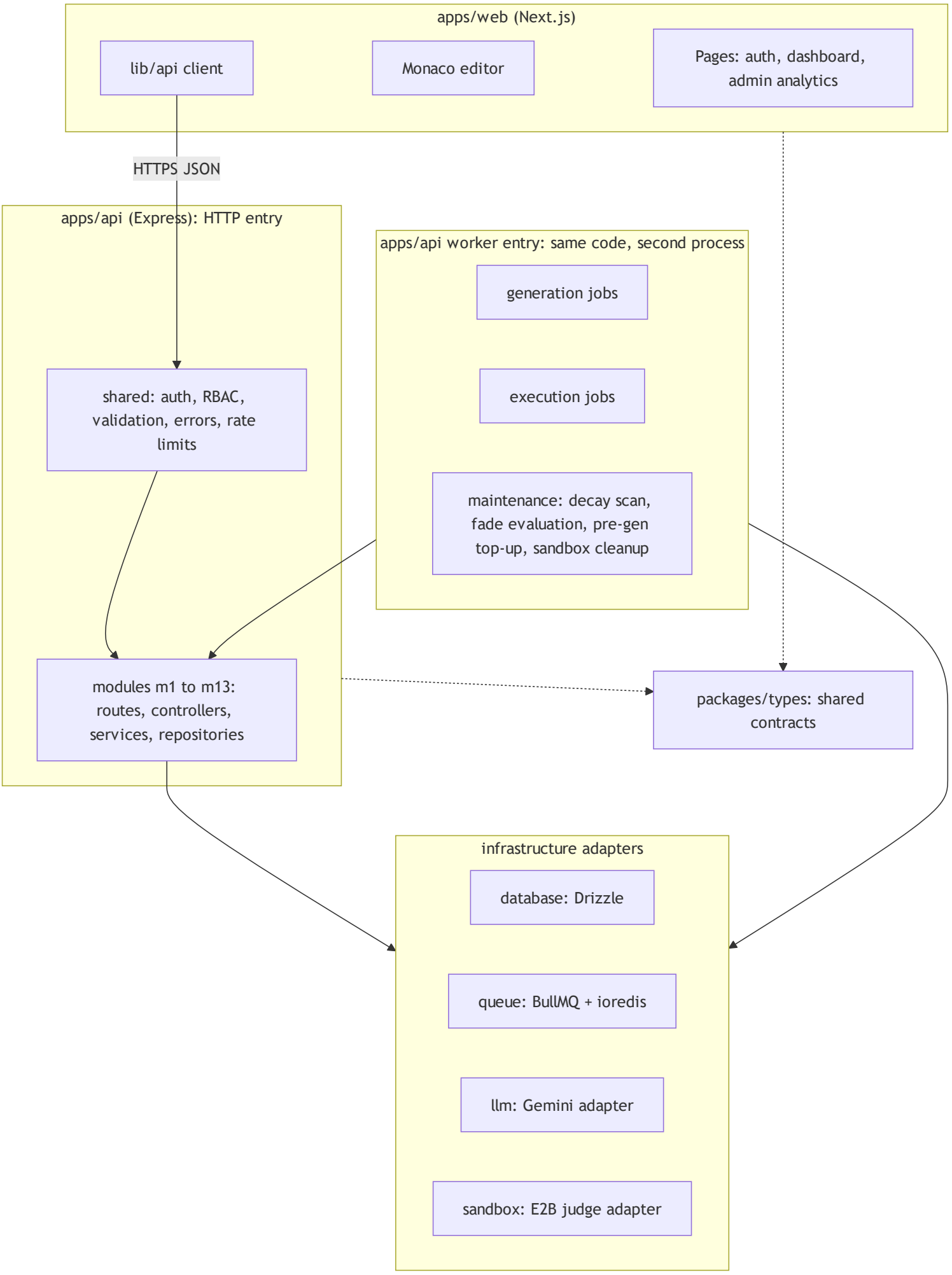


6.10 Activity diagram: next-step selection (M5)

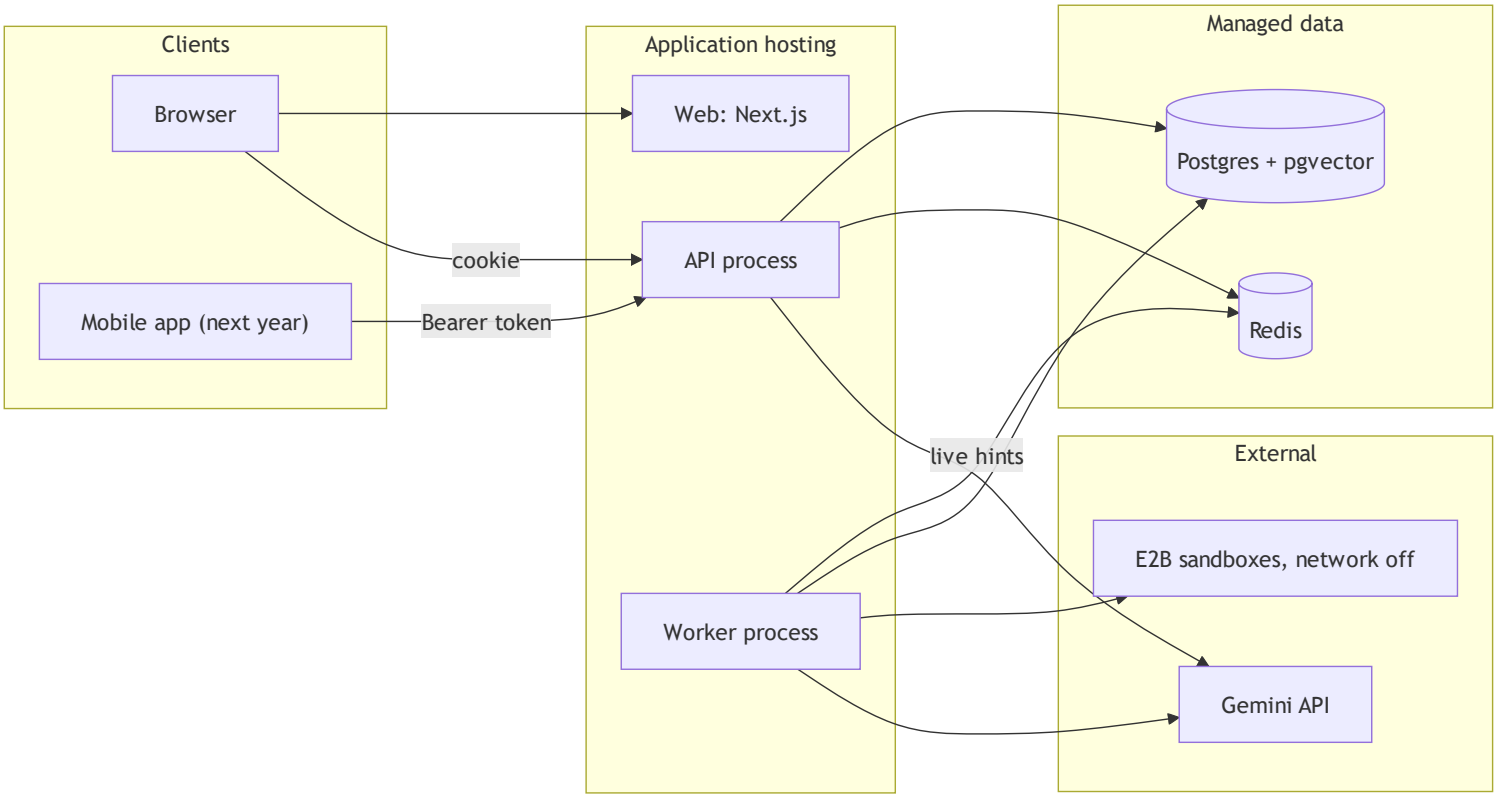

Start
Load mastery, reviews, graph, fade stage
Compute effective mastery for each skill
Any reviews due?
yes
no
Add due reviews to candidates
Add skills whose prerequisites are all above threshold
Any candidates?
no
yes



6.11 Component diagram



6.12 Deployment diagram



In development, Postgres and Redis run locally through `docker compose`.

---

7. API design

7.1 Conventions

Item	Convention
Base path	`/api/v1`
Format	JSON over HTTPS
Auth	httpOnly cookie (web) or `Authorization: Bearer` (mobile); role checked on every author/admin route
Validation	zod schema per request; shared types in `packages/types`
Success	`{ "success": true, "data": {...}, "meta": {...} }`
Error	`{ "success": false, "error": { "code", "message", "details" } }`
Slow operations	`202 Accepted` + an ID; the client polls for the result

Item	Convention
Internal operations	Called by services only, never exposed to the client (e.g. recording evidence)

7.2 MVP APIs (10 Nov)

Module	Endpoints
M1 Auth	`POST /auth/register` · `POST /auth/login` · `GET /auth/me`
M3 Diagnostic	`POST /diagnostic/start` · `POST /diagnostic/:id/answers` · `POST /diagnostic/:id/complete`
M4 Mastery	`GET /mastery` · `GET /mastery/:skillId` · evidence recording is internal only
M5 Recommender	`GET /recommendations/next`
M6 Exercises	`GET /exercises/:id` · `POST /exercises/generate` · `GET /exercises/generation/:jobId`
M7 Submissions	`POST /submissions` · `GET /submissions/:id/result`
M8 Hints	`POST /exercises/:id/hints`
M9 Sessions	`POST /sessions` · `GET /sessions/active`
M11 Progress	`GET /progress`
M12 Fade	`GET /scaffolding/state`
M13 Analytics	`GET /analytics/overview` (admin)

20 client-facing endpoints, plus internal evidence recording. The full plan has ~98 operations; every one maps onto the tables in section 5:

Module	Ops	Tables
M1	8	users, organizations, refresh_tokens, password_reset_tokens
M2	12	skills, skill_prerequisites, skill_graph_changes
M3	6	diagnostic_*, enrollments
M4	9	mastery_records, evidence_events

Module	Ops	Tables
M5	6	recommendations
M6	9	exercises, exercise_tests, exercise_hints, generation_jobs, generation_failures
M7	7	submissions, sandbox_runs, run_test_results, exercise_attempts
M8	7	hint_events, exercise_hints, exercise_attempts
M9	8	learning_sessions, session_steps
M10	6	review_items
M11	6	reads mastery_records, evidence_events, skills
M12	6	fade_schedules, fade_stages, learner_scaffolding, fade_stage_changes
M13	8	calibration_records, llm_calls, analytics views

8. Non-functional design

Requirement	How the design meets it
NFR-P1 Hint latency	Hints are pre-generated at freeze time; live generation uses the small model only when no stored hint exists
NFR-P2 Generation time	Generation runs as a background job; the learner is served a ready or template exercise
NFR-S1 Sandbox	Code runs only in E2B with network off and time and memory limits
NFR-S2 Hint enforcement	Hint level and ceiling are server-side columns, updated under a row lock
NFR-S3 Passwords	argon2id hashes only; refresh tokens also stored as hashes
NFR-S4 Role checks	RBAC middleware on every author/admin route, not just in the UI
NFR-R1 Generation failure	Automatic fallback to the template bank, no error shown
NFR-R2 Sandbox vs code failure	Separate SE verdict; SE never affects mastery

Requirement	How the design meets it
NFR-U1 / U2 Accessibility	Monaco keyboard support; progress view uses labels and values, not colour alone
NFR-M1 Subject independence	Mastery and recommender work on generic skills; course and language are data, not code
NFR-SC1 Scalability	Generation and execution go through BullMQ to a separately scaled worker process

Determinism. Content is generated once, verified and frozen, then served by its content key. Grading uses only sandbox execution with a fixed compiler and seeded test generators. Next-step selection is rule-based with a fixed tie-break, and every recommendation and evidence row records the algorithm or formula version used. No LLM output sits in the grading or selection path.

Security beyond the NFRs. Secret exercise fields (solutions, harness, hidden tests) are stripped from every response. Rate limits in Redis protect login, submissions, hints and generation. Learner code reaches Gemini only inside live hint prompts, which is covered by recorded consent.

---

9. Requirement traceability

SRS requirement	Design element
FR-M1-01 Register with unique ID	`users.email` UNIQUE, sequence 6.4
FR-M1-02 Sessions per user type	JWT + `refresh_tokens`, `users.role`
FR-M1-03 Record goal	`enrollments.goal_skill_id`, `goal_text`
FR-M1-04 Role-based access	RBAC middleware, route groups in web
FR-M1-05 Password reset (Should)	`password_reset_tokens` (after MVP)
FR-M2-01 Skills as DAG	`skills`, `skill_prerequisites`
FR-M2-02 Reject cycles	Cycle check in M2 service + self-loop CHECK

SRS requirement	Design element
FR-M2-03 Editing screen	**MVP: seeded by script.** Screen after MVP; APIs designed (M2, 12 ops)
FR-M2-04 Syntax vs concept type	`skills.skill_type`, `decay_lambda`
FR-M2-05 Graph history (Could)	`skill_graph_changes`
FR-M3-01 Adaptive test	`diagnostic_items` by difficulty and dimension
FR-M3-02 Seed mastery	`diagnostic_results` → `mastery_records`
FR-M3-03 Retake	`diagnostic_attempts.attempt_no`
FR-M4-01 Mastery fields	`mastery_records`
FR-M4-02 Evidence only	DFD level 1: only process 4 writes D4; `evidence_events`
FR-M4-03 Exponential decay	Effective mastery computed on read
FR-M4-04 Revision queue	`review_items` created below threshold
FR-M4-05 Weighted evidence	`evidence_events.weight` from rung, hints, time
FR-M5-01 to 05 Recommender	Activity diagram 6.10, `recommendations.score_breakdown`, no LLM in the flow
FR-M6-01 Generate	Generation pipeline, DFD 4.4, sequence 6.8
FR-M6-02 Verify before use	Reference run + mutation check, exercise state diagram
FR-M6-03 Template fallback	`exercises.source = template`, process 6.7
FR-M6-04 Log failures	`generation_failures`
FR-M7-01 Isolated sandbox	E2B, network off
FR-M7-02 Per-test results	`run_test_results`
FR-M7-03 Limits reported separately	TLE and MLE verdicts, mutation check
FR-M7-04 Structured result to M4, M9	Sequence 6.5: evidence + session step update

SRS requirement	Design element
FR-M8-01 Hint level on server	`exercise_attempts.hint_level`
FR-M8-02 Only current level sent	Sequence 6.6
FR-M8-03 Advance only on request	Hint ladder state diagram; CE ×3 only *offers*
FR-M8-04 Output filter	`OutputFilter`, `hint_events.filter_result`
FR-M8-05 Log bottom-out	`hint_events` kind `bottomed_out`, `exercise_attempts.bottomed_out`
FR-M9-01 Start, pause, resume, end	Session state diagram
FR-M9-02 Steps from recommender	`session_steps.recommendation_id`
FR-M9-03 Resume at same step	`learning_sessions.current_step_id`
FR-M10-01 Revision date from decay	`review_items.due_at`, scheduled decay scan
FR-M10-02 Queue visible to M5	Recommender reads `review_items`
FR-M11-01 to 03 Honest progress view	Reads stored mastery, reading and writing kept separate
FR-M12-01 Configurable schedule	`fade_schedules`, `fade_stages`
FR-M12-02 Enforce limits	`hint_ceiling`, `min_rung` clamp in M5 and M8
FR-M13-01 Calibration	`calibration_records`, holdout exercises
FR-M13-02 Hint trends	Views over `hint_events`
FR-M13-03 Generation rates	Views over `generation_jobs`, `generation_failures`

10. Partial implementation status

Done (in the repository):

- pnpm monorepo: `apps/api` (Express + TypeScript), `apps/web` (Next.js), `packages/types`, `packages/config`.
- Module folders for M1–M13 with routes, controllers and services layers.
- Local infrastructure via `docker compose`: PostgreSQL 16 with pgvector, Redis 7, with secrets in `.env`.
- API server starts, serves `GET /health`, and connects to PostgreSQL through a pooled `query()` helper.
- Shared TypeScript configuration and initial shared types.

Phase 2 slice (in progress):

- Drizzle migration for `organizations`, `users`, `refresh_tokens`.
- `POST /auth/register`, `POST /auth/login`, `GET /auth/me` with argon2id hashing and JWT cookie/Bearer auth.
- Shared error handling, response envelope and request validation.

Next (Phase 3, 23 Oct): skill graph seed, enrollment and diagnostic, mastery engine, first E2B judge run (AC/WA/CE/TLE end to end), API contract frozen for the frontend.

---

11. Open items

Item	Needed by
M4 update formula: size of AC gain and WA penalty	Mastery engine (Phase 3)
Prerequisite threshold (0.7 is a placeholder)	Recommender
Final fade-stage values; whether a stage can step back	M12
Template exercises per skill and rung	M6 fallback
Session idle and timeout values	M9
Hosting provider within budget	Deployment
Retention period for learner code after the study	Research trial